ABSTRACT

Title of Thesis: A FRAMEWORK FOR BENCHMARKING GRAPH-BASED ARTIFICIAL INTELLIGENCE

Kent Daniel O'Sullivan
Master of Science, 2024

Thesis Directed by: Professor William Regli
Department of Computer Science

Graph-based Artificial intelligence (GraphAI) encompasses all the AI problems that we formulate as graphs, that operate on graphs, or that rely on the structural characteristics of graphs for learning. GraphAI is becoming increasingly important with the emergence of generative AI as a mechanism for grounding generation in curated, traceable knowledge and improving transparency, explainability and performance. GraphAI is important in many real-world applications beyond abstract generative use cases, with a cursory survey of national security problems highlighting the ubiquitous need to convert unstructured data to graphs, analyze their content and structure and build predictive models from them.

Since GraphAI requires combining different representations, algorithms, and systems, it is particularly hard to characterize good GraphAI performance whether in development or when deployed to solve problems in the real world. Clear comparison of performance is hard because as graphs vary in size and shape, they trigger unpredictable changes in computing, memory, and network usage. The unpredictable results of combining different representations, algorithms

and systems for GraphAI are exacerbated by a broad diffusion of abstractions and programming models further obfuscating what characterizes 'good' performance in GraphAI .

Existing Benchmarks address some of these issues by providing collections of shared datasets, identifying the most fundamental tasks for comparison and enabling the comparison of relative execution time. However, current approaches are not sufficient to capture the breadth of complexity needed to characterize GraphAI . Datasets are sorely lacking in coverage of weighted, directed, attributed multigraphs characteristic of knowledge graphs that form the basis of most modern GraphAI inputs. There is almost no coverage of dynamic (streaming) graph tasks, and no work approaching GraphAI problems within size, weight and power constraints.

Metrics are limited to efficiency measures like execution time, largely ignore graph extraction, transformation and load times, and either ignore the correctness of the output or cost cost of generating it.

Using an empirical approach, we define a framework for constructing a comprehensive GraphAI benchmark that addresses the shortcomings outlined above. We propose a taxonomy of GraphAI problems and describe how to select appropriate tasks, datasets, and metrics given a use case or problem under study.

To demonstrate the suitability of our approach, we select the problem of *Population Segmentation* using the *structural analysis* GraphAI task of *Community Detection* from our survey of national security problems as a case study, and develop a benchmark that provides a researcher the information they need to trade-off between efficiency, effectiveness and cost.

The case study shows how the methodical construction of the benchmark using empirical methods yields an effective benchmark for *Community Detection* and a viable template for implementing other GraphAI benchmarks as part of the suite in the future.

A FRAMEWORK FOR BENCHMARKING GRAPH HARDWARE ACCELERATORS

by

Kent Daniel O'Sullivan

Thesis submitted to the Faculty of the Graduate School of the
University of Maryland, College Park in partial fulfillment
of the requirements for the degree of
Master of Science
2024

Advisory Committee:
William Regli, Chair/Advisor
Mohammad Hajiaghayi
Brian Pierce
Laxman Dhulipala

© Copyright by
Kent O'Sullivan
2024

Acknowledgments

TBC Following acceptance of Thesis

Table of Contents

Acknowledgements ii

Table of Contents iii

List of Tables vi

List of Figures vii

List of Abbreviations viii

Chapter 1: Introduction 1
- 1.1 Graph-Based Artificial Intelligence . . . 2
  - 1.1.1 A Taxonomy of GraphAI . . . 2
  - 1.1.2 Real-World Uses of GraphAI . . . 4
  - 1.1.3 GraphAI Challenges . . . 6
  - 1.1.4 GraphAI Way Forward . . . 6
- 1.2 Problem . . . 7
- 1.3 Thesis Outline . . . 8

Chapter 2: Preliminaries 10
- 2.1 Graph Definitions . . . 10
- 2.2 Graph Topology . . . 13
  - 2.2.1 Real-World Graphs . . . 14
  - 2.2.2 Synthetic Graphs . . . 15
  - 2.2.3 Summary . . . 16
- 2.3 Graph Representations . . . 16
  - 2.3.1 Graph Storage . . . 17
  - 2.3.2 Graph Abstractions . . . 21
- 2.4 Graph Primitive Operations . . . 23
- 2.5 Architectural Constraints for Graph Processing: Locality . . . 24
  - 2.5.1 Locality . . . 25

Chapter 3: Literature Review 29
- 3.1 Benchmarking . . . 29
  - 3.1.1 Competitive Benchmarks . . . 30
  - 3.1.2 Empirical Benchmarks . . . 31
  - 3.1.3 Constrained Benchmarks . . . 32

3.1.4 Summary . . . 32
3.2 Benchmarking GraphAI . . . 33
3.2.1 Task Coverage. . . . 35
3.2.2 Dataset Coverage. . . . 38
3.2.3 Metric Coverage. . . . 41
3.3 GraphAI Bottlenecks . . . 43
3.3.1 Memory Bottlenecks . . . 43
3.3.2 Computation Bottlenecks . . . 44
3.3.3 Communication Bottlenecks . . . 45

Chapter 4: The Graph Artificial Intelligence Benchmarking Framework 46
4.1 Motivation . . . 46
4.2 Background . . . 47
4.2.1 Benchmark Design Principles . . . 47
4.2.2 Components of a Benchmark . . . 48
4.3 Design . . . 51
4.3.1 Empirical Benchmark Design . . . 52
4.4 Abstract Framework . . . 54

Chapter 5: The Population Segmentation Problem 61
5.1 Identify and Characterize the Population Segmentation Problem . . . 61
5.1.1 Definition . . . 61
5.1.2 Tasks . . . 62
5.1.3 Datasets . . . 63
5.1.4 Outputs . . . 64
5.2 Identify and Characterize Phenomena of Interest . . . 64
5.2.1 Efficiency . . . 65
5.2.2 Effectiveness . . . 65
5.2.3 Cost . . . 66
5.2.4 Outputs . . . 66
5.3 Conduct Exploratory Experimentation . . . 67
5.3.1 Datasets Observations . . . 69
5.3.2 Implementation Observations . . . 69
5.3.3 Outputs . . . 71
5.4 Develop Hypotheses . . . 72
5.4.1 Efficiency Hypotheses . . . 73
5.4.2 Effectiveness Hypotheses . . . 73
5.4.3 Constraint Hypotheses . . . 73
5.4.4 Outputs . . . 74
5.5 Construct the Investigation Apparatus . . . 74
5.5.1 Observation Apparatus . . . 75
5.5.2 Datasets . . . 75
5.5.3 Tasks . . . 80
5.5.4 Metrics . . . 80
5.5.5 Limitations . . . 81

5.5.6 Outputs 81
5.6 Analyze Results, Develop Theoretic Explanation and Iterate 81
5.6.1 Demonstration Setup 81
5.6.2 Demonstration Results 84
5.7 Discussion 87

Chapter 6: Conclusion **89**
6.1 Future Work 89
6.2 Conclusion 90

Bibliography **93**

List of Tables

1.1 A brief taxonomy of GraphAI Problems . . . 3
1.2 A Survey of GraphAI Problems in the Intelligence & Security Domain . . . 5

3.1 Coverage of Graph-Based AI tasks by Benchmark Suites and Workload Analyses (†) ‡=Inferred, not specified. . . . 34
3.2 Coverage of Graph-Based AI Datasets by Benchmark Suites and Workload Analyses (†) . . . 37
3.3 Coverage of Graph-Based AI Metrics by Benchmark Suites and Workload Analyses (†) . . . 40

4.1 Summary of dependencies between benchmark components and evaluation metrics 51
4.2 Summary of expected metric behaviors (↑ Improve, — Stable, ↓ Degrade) when a benchmark component is constrained . . . 58

5.1 Example Problems for Population Segmentation . . . 63
5.2 Summary of Computational Approaches to Population Segmentation . . . 68
5.3 Summary of Datasets for the Population Segmentation problem . . . 79
5.4 Summary of Experiments for CD Demonstration . . . 83

List of Figures

2.1 Example graph with five nodes and five edges . . . 17
2.2 Adjacency List for graph in figure 2.1. Each line represents a vertex with the first in the list being the source and each subsequent vertex a destination of an edge. . 18
2.3 Adjacency Matrix for graph in figure 2.1 is a $|V| \times |V|$ matrix where a value of 1 indicates an edge and 0 is no edge. . . . 19
2.4 Incidence Matrix for graph in figure 2.1 is a $|E| \times |V|$ matrix where a -1 is the source node and 1 is the destination of a given edge. . . . 20
2.5 The Compressed Sparse Row (CSR) format uses 3 lists to represent the graph in figure 2.1. . . . 21
2.6 A simplified representation of a direct graph in vertex-order where each vertex is added to memory in the order it was created (here, alphabetically). Edges are not represented here. . . . 26
2.7 A simplified representation of a direct graph in vertex order where each vertex is added to memory in the order it was created with attributes stored separately. Pointers from topology to attributes not shown for clarity, but correspond to first letter of attribute. . . . 27

4.1 The basic components of a generic benchmark system . . . 49
4.2 Hooker's methodology for an empirical science of algorithms [1, 2] . . . 53
4.3 The GraphAI Benchmark Framework is a collection of empirically developed problems that share a common observation apparatus for timing and evaluation. . 55
4.4 GraphAI Benchmark Design Framework . . . 60

5.1 Community Detection Problem Apparatus . . . 82
5.2 Efficiency results for Community Detection show quadratic time complexity. . . . 84
5.3 Execution time dominates the Louvain approaches. . . . 85
5.4 NMI appears disconnected from predicting the correct number of communities. . 85
5.5 The incremental approach is erratic, varying the number and quality of communities at each iteration. . . . 86
5.6 NMI varies at each iteration with no interim step being significantly different from the result at termination . . . 86

List of Abbreviations

A*	A-Star Search [Algorithm]
AI	Artificial Intelligence
ARLIS	Applied Research Laboratory for Intelligence and Security
BC	Betweenness Centrality
BFS	Breadth First Search [Algorithm]
BLAS	Basic Linear Algebra Subprograms
BSP	Bulk-Synchronous Parallel [programming model]
CC	Connected Components
CRUD	Create, Read, Update, Delete (database operations)
CSC	Compressed Sparse Column
CSP	Constraint Satisfaction Problem
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DARPA	Defense Advanced Research Projects Agency
DCM	Data-Centric Model for programming
DoD	United States Department of Defense
DFS	Depth First Search [Algorithm]
DRAM	Dynamic Random Access Memory
DSL	Domain Specific Languages
FLOPs	Floating Point Operations
FLOPS	Floating Point Operations per Second
FLOPS/W	Floating Point Operations per Second per Watt
GA	Graph Abstraction [AI problem category]
GAP	Berkeley Graph Algorithm Platform
GAS	Gather-Apply-Scatter [programming model]
GBBS	Graph-Based Benchmark Suite
GCC	Global Clustering Coefficient [Algorithm]
GCN	Graph Convolution Network
GDB	Graph Database
GNSS	Global Navigation Satellite System
GNN	Graph Neural Network
GP	Graph Prediction [AI problem category]
GR	Graph Reasoning [AI problem category]
HIVE	Hierarchical Identify Verify Exploit
HVT	High Value Target
IIoT	Industrial Internet of Things

I/O	Input/Output
IoT	Internet of Things
IPC	Instructions per Cycle
I&S	Intelligence and Security
KG	Knowledge Graph
LCC	Local Clustering Coefficient [Algorithm]
LLC	Last Level Cache
MLCC	Mean Local Clustering Coefficient [Algorithm]
MIT	Massachusetts Institute of Technology
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
MST	Minimum Spanning Tree [Algorithm]
NMI	Normalized Mutual Information
NUMA	Non-Uniform Memory Access
OE	Operating Environment
OPSEC	Operational Security
OSINT	Open Source Intelligence
OSM	Open Street Maps
POI	Point of Interest
PR	PageRank [Algorithm]
RAM	Random Access Memory
SA	(Graph) Structural Analysis [AI problem category]
SCC	Strongly Connected Components [Algorithm]
SGM	(isomorphic) Sub Graph Matching [Algorithm]
SNAP	Stanford Network Analysis Project
SOTA	State of the Art
SPEC	Standard Performance Evaluation Corporation
SPM	Spatial Pattern Matching
SPS	Searching, Planning and Scheduling [AI problem category]
SSSP	Single Source Shortest Path [Algorithm]
STGNN	Spatio-Temporal Graph Neural Network
SWaP	Size Weight and Power
TAI	Trustworthy AI [AI problem category]
TB	Terabytes
TC	Triangle Counting [Algorithm]
TEPS	Traversed Edges per Second [Metric]
TEPS/W	Traversed Edges per Second per Watt
UCS	Uniform Cost Search [Algorithm]
WCC	Weakly Connected Components [Algorithm]

Chapter 1

Introduction

Graph data structures and the algorithms that operate on them are foundational to computer science. As the social media platforms of the early 21st century have proliferated into the global consciousness, graphs have been increasingly used as powerful abstractions to represent interconnected data collected from the real world. Recent years have seen a 'Cambrian explosion' of artificial intelligence (AI) applications, and the hardware accelerators used to improve their efficiency [3]. Tasks that have traditionally been in the domain of graph processing are being challenged by modern machine-learning approaches to graphs. We term the overlap of graph processing and AI and GraphAI .

For GraphAI to be useful in the real world, systems must be capable of processing massive, heterogeneous, dynamic graphs under resource constraints. Maximizing performance given limited resources involves making an informed tradeoff between these factors, which requires having a mechanism for evaluating GraphAI systems under varying conditions.

Benchmarks are widely used, effective tools to compare the relative performance of implementations and the environments that they run in. However, they tend to manifest as platforms for 'racing' systems against each other rather than tools for scientific inquiry. Developing a comprehensive benchmark requires a rigorous framework, which is the goal of this thesis.

The remainder of Chapter 1 will describe GraphAI , illustrating the taxonomy we propose using a study of GraphAI in the *Intelligence and Security* (I&S) domain. The chapter concludes by formulating the problem of how to measure the tradeoffs in effectiveness and efficiency in

GraphAI systems.

1.1 Graph-Based Artificial Intelligence

1.1.1 A Taxonomy of GraphAI

Graph-Based Artificial Intelligence (GraphAI) is a sub-field of computer science dedicated to developing models and algorithms to support learning and reasoning over information represented in graph data structures. Graphs are powerful abstractions of the real world that capture how entities relate to each other across a variety of dimensions, including social, spatial, semantic and logical. While graphs can express a wealth of information about the world, applying AI to that information efficiently and effectively is challenging.

We divide GraphAI into a taxonomy covering several overarching "AI Problem" categories (*Searching, Planning & Scheduling; Structural Analysis; Reasoning; Graph Abstraction; Graph Prediction* and *Trustworthy AI*), which are outlined in Table 1.1. Each AI problem category is associated with a few key GraphAI problems that each involve low-level graph processing tasks. We include a few typical examples of such tasks for each GraphAI problem to convey the breadth of tasks that a GraphAI benchmarking framework must cover. In Chapter 4 we discuss the challenges associated with designing a benchmark that can measure performance across all GraphAI tasks.

AI Problem	GraphAI Problem	Example Tasks
Searching, Planning & Scheduling	Search	BFS, DFS, UCS, A*, MinMax
Formulate as graph problem [4]	Constraint Satisfaction	Forward Checking, Backtracking
	Planning	GraphPlan, SSSP, MST
	Scheduling	CriticalPath
	Inference	Variable Elimination (Bayes Net)
	Reinforcement Learning	Q-Learning
Structural Analysis	Pattern Matching	Subgraph Matching, Spatial Pattern Matching
Operate on graph topology [5–11]	Characterization	WCC, SCC, LCC, GCC, TC
	Graph Clustering	Community Detection
	Graph Partitioning	Modularity Optimization
	Influence Mapping	PageRank, BetweennessCentrality
Graph Reasoning	Knowledge Acquisition	Knowledge Graph Completion, Ontology Learning
Operate on graph knowledge [12–16]	Logic-Based Reasoning	Logical Inference, Ontological Inference
	Representation Reasoning	Tensor Factorization, Semantic Matching
	Neural Reasoning	Relation Inference, Entity Resolution
	Natural Language Processing	Question Answering, Textual Entailment
Graph Abstraction	Topological Representation Learning	Node2Vec, DeepWalk
Embed for downstream tasks [12, 17, 18]	Knowledge Representation Learning	Complex Vector Embeddings
Graph Prediction	Graph Classification	Node, Edge & Graph Classification
Learn directly on graphs [19–21]	Graph Completion	Link & Property Prediction, Knowledge Acquisition
	Dynamic Graph Prediction	Traffic Flow Prediction
Trustworthy AI	Grounding	NeuroSymbolic Systems
Explainability & Interpretability [22–28]	Knowledge Structuring	Retrieval Augmented Generation
	Output Interpretation	Post-Hoc Explanation

Table 1.1: A brief taxonomy of GraphAI Problems

1.1.2 Real-World Uses of GraphAI

GraphAI is used heavily in real-world problem-solving.

A recent survey of the problems, datasets and challenges faced by *Intelligence and Security* (I&S) professionals by the *Applied Research Laboratory for Intelligence and Security* (ARLIS) [29] shows that most analytical problems faced by respondents are some form of network (graph) analysis. Further, most computational I&S problems are consistent with the irregularity inherent to graph processing, and cannot be solved easily by distributing the computing or installing general-purpose accelerators like Graphics Processing Units (GPU).

That is, most of the problems faced by the I&S community suffer from the computational limits of GraphAI , and so improving the efficiency and effectiveness of GraphAI is critical to improving the ability of the I&S community to solve problems. We broaden the scope of that survey by conducting a literature review of I&S problems across geospatial, targeting, cyber security, financial and incident response to find that many of the problems are either graph-native or can be represented with graphs and analyzed using GraphAI .

We break the problem space down by the AI problems from our taxonomy. For each I&S problem, we identify the expected data type, likely GraphAI approach to solving the problem and key constraint in table 1.2. We find large streaming knowledge graphs are a common input format, and that there are frequently temporal constraints that preclude GraphAI approaches that cannot produce solutions in minutes to hours.

AI Problem	I&S Problem	Input Data	GraphAI Approach	Constraints
SPS	Link Analysis [30]	Knowledge Graph	Reachability	Heterogeneity
	Navigation	Spatial graph	Path Planning	SWaP; Complexity
SA	HVT Identification	Multi-Source Graphs	Influence Mapping	Heterogeneity
	Co-Traveller Detection [29, 31–34]	Spatio-Temporal Data	Clustering	Temporal
	POI Identification [6, 35]	Location Data	Spatial Pattern Matching	Complexity
	Demographic Segmentation [36–38]	Communication Logs, Social Media	Community Detection	Temporal
	Counter-Threat Finance [39–43]	Transaction Records, Blockchain	Subgraph Matching	Temporal
Reasoning	Kill-Chain Analysis [44–47]	Event Logs	Logical Inference	Completeness
	All-Source Fusion [29, 48, 49]	Streaming Knowledge Graph	Entity Resoultion	Temporal
Prediction	Anomaly Detection [50, 51]	Streaming Knowledge Graph	Classification	Temporal
	Link Prediction [52–55]	Streaming Knowledge Graph	Completion	Temporal

Table 1.2: A Survey of GraphAI Problems in the Intelligence & Security Domain

1.1.3 GraphAI Challenges

Developing efficient and effective GraphAI is challenging because of the *massive, heterogeneous* and increasingly *dynamic* nature of input data. From the problem survey, we identify a requirement to support graph processing at the billions-of-edges level on heterogeneous, dynamic graphs. To make the problems more tractable, simplification approaches to the data like sampling and dimensionality reduction trade-off completeness of information to gain processing speed. The trade-offs become more pronounced in resource-constrained settings when time, size, weight and power are limited, users may accept a reduction in one dimension to gain in another. The willingness to trade off effectiveness for efficiency for complex problems and when under externally imporsed constraints is the precedent that our approach in Chapter 4 leverages for constructing the benchmark.

1.1.4 GraphAI Way Forward

To begin to address the challenges outlined above, we need a mechanism to compare GraphAI implementations and the systems they run on. *Benchamrks* and *Workload Analyses* are the two most widely used comparison mechanisms.

Benchmarks. Benchmarks provide an objective mechanism to judge the relative performance of candidates on a representative problem. They are usually targeted to reduce uncertainty about how long a problem will take to solve for a general audience and thus focus on simple metrics. The focus on execution time is typically beneficial as it is usually the best indicator of a system's performance [56]. Current benchmarks for GraphAI programs are almost exclusively focused on simplified, static graphs and generally exclude Extract, Transform and Load (ETL)

time Benchmarks rarely report the *effectiveness* metrics along with the *efficiency* metrics, making them unsuitable for evaluating approximate or incremental approaches. As a result, benchmarks provide some useful insight, but to robustly compare GraphAI requires an approach that goes beyond 'racing' systems against each other.

Workload Analyses. Workload analyses provide a more robust analysis of a given implementation in a particular environment on a particular workload. They are typically geared towards identifying bottlenecks and informing future development to exploit or elevate those bottlenecks. When workload analyses are executed on a real machine, the characterization typically requires code instrumentation and invasive system-level profiling to collect the metrics required to identify the bottlenecks. As a result, most of the benefit from workload analyses comes from varying the programming model, hardware allocation, or graph representation on a single system, since it is difficult to compare metrics collected across systems.

The challenge moving forward is developing a path that combines the usability of a general benchmark with the informativeness of a workload analysis without oversimplifying the analysis or needing invasive access to any system we wish to test.

1.2 Problem

GraphAI covers a broad collection of tasks with varying computational requirements. Most organizations that employ GraphAI will need to solve problems using tasks from across the GraphAI taxonomy. Selecting the optimal system configuration, hardware accelerators and implementations of GraphAI tasks is essential to effectively addressing the breadth of real-world problems, particularly when faced with time, cost and other constraints. Organizations may be

required to trade-off between the effectiveness, efficiency and cost of solutions, and need to understand what the implications of those tradeoffs are.

Competitive benchmarks and workload analyses offer two paths to understand these tradeoffs, but either provide too little information or require invasive instrumentation to characterize the System Under Test (SUT).

This thesis proposes a hybrid solution to the problem of GraphAI benchmarking, where we draw from both existing benchmarking and workload analysis studies to develop an approach that allows for an investigator to trade-off between effectiveness and efficiency when testing a system. We develop a benchmarking framework for GraphAI to bridge the gap between competitive benchmarks and invasive workload analyses that limits access to the Input/Output interface and maximizes the freedom of a user to modify the system under test. Our approach is problem-centric, system-agnostic and gives the user a mechanism for understanding of how system and implementation changes impact the efficiency, effectiveness and cost of their system. The design draws on the empirical study of algorithms, constraint theory and the significant body of knowledge in benchmarking and analyzing workloads of GraphAI to develop a formal methodology for developing GraphAI benchmark problems.

1.3 Thesis Outline

The thesis continues with a review of key graph and computer architecture concepts in Chapter 2 to identify what it is that makes optimizing GraphAI approaches hard and why measuring relative performance is difficult. Then, Chapter 3 is a detailed review of current GraphAI benchmarking and workload analysis efforts to characterize the current gaps in benchmark coverage

of GraphAI tasks, datasets and metrics and identify what causes performance to degrade in GraphAI systems. Chapter 4 introduces the GraphAI benchmark frameworks and outlines our methodology for developing GraphAI problems to constitute the benchmark framework. We use the population segmentation problem from the Intelligence and Security domain to demonstrate how our development methodology is applied, describe the implementation of the GraphAI benchmark prototype and outline the findings of our initial experiments in chapter 5. Finally, chapter 6 concludes after identifying some specific directions for future work.

Chapter 2

Preliminaries

In this chapter, we introduce the core definitions and building blocks of graph processing that are central to GraphAI and review key concepts in computer architecture that contibute to current GraphAI system bottlenecks.

2.1 Graph Definitions

Graphs are the fundamental data structure common to GraphAI problems. Below we define the components of a graph and discuss some of the core properties a graph can possess.

Definition 0 [Graph (G)]: The data structure comprising of the related vertices and edges such that $G = \{V, E\}$. Also called a 'network'. $|G|$ represents the size of the graph, and when used in this general sense it means the sum of the number of vertices and edges.

Definition 1 [Vertex (V)]: Represents an 'entity' or 'thing'. Also called a node. $|V|$ is the number of vertices (size of the vertex set).

Definition 2 [Edge (E)]: Represents a relationship between two vertices. Also called arcs or links. $|E|$ represents the number of edges (size of the edge set).

Definition 3 [Attribute]: A label, property or other information that is encoded in a vertex or edge of a graph. Graphs with attributes are called attributed graphs.

Definition 4 [Weight]: A numerical value assigned to an edge to describe some real-world value it represents, like the number of citations, length of a road, or cost of an action. Graphs with weights are called weighted graphs.

Definition 5 [Direction]: The flow of information across an edge from the source node to the destination node. Graphs having edges with direction are called directed graphs.

Definition 6 [Multi-Edge]: A special case of directed edge where more than one edge may exist between a given pair of vertices. Graphs with multi-edges are called multi-graphs and are typical of heterogeneous graphs like knowledge graphs.

Definition 7 [Hyper-Edge]: An edge that connects an arbitrary number of vertices. Graphs with hyper-edges are called hyper-graphs.

Graphs have the following characteristics that define their size and shape, including their topology, degree distribution and diameter.

Definition 8 [Topology]: The structural characteristics of a graph that emerge when edges link vertices together, regardless of attributes or edge weights. Common topologies include mesh, scale-free and uniform-random.

Definition 9 [Degree]: The count of edges at a given vertex. Frequently separated into degree$_{out}$ and degree$_{in}$ in directed graphs.

Definition 10 [Average Degree]: The mean degree across all vertices in the graph. A higher mean degree indicates a denser graph.

Definition 11 [Degree Distribution]: The distribution of vertex degrees in a given graph. Can follow any statistical distribution and correlates strongly with known graph processing bottlenecks.

Definition 12 [Neighborhood]: The collection of vertices that have edges directly connecting them to a given vertex. The K-hop neighborhood is the collection of vertices within K edges of a given vertex for example a 'friend of a friend'.

Definition 13 [Diameter]: The length of the shortest path between the two most distant vertices in a graph, where distance is measured by the number of hops between vertices. Sparser graphs tend to have larger diameters because there are fewer high-degree nodes to bridge gaps between otherwise distant nodes.

Although graphs can have a variety of characteristics, they can be broadly categorized as simple or heterogeneous, and static or dynamic.

Definition 14 [Simple Graph]: The most basic form of a graph. It may be directed, weighted,

and attributed, but the attributes are of a consistent type (e.g. all vertex labels are of type 'name').

Definition 15 [Heterogeneous Graph]: A graph with multiple 'types' of vertex, such as a social graph that has vertices to represent 'person' and vertices to represent 'organization'. Heterogeneous graphs are more complicated to process than simple graphs because they require type-checking before acting on a given vertex or edge.

Definition 16 [Static Graph]: A graph whose vertices, edges, weights, and attributes are known apriori and do not change. Static graphs are generally simpler to process than dynamic graphs.

Definition 17 [Dynamic Graph]: A graph that evolves with its vertices, edges, attributes, and weights being created, updated, and deleted over time. Also called a streaming graph. Dynamic graphs introduce temporal constraints on graph processing that make many of the well-researched static approaches unviable.

2.2 Graph Topology

Graphs fit into core classes (scale-free, mesh, random) based on their size, shape, topological structure, and construction method. We begin by addressing the classes of graphs that occur in the real world, and then discuss synthetically generated graphs, which can be constructed randomly or using a prior distribution.

2.2.1 Real-World Graphs

This section will first identify the 'real world' graph classes that are derived from real-world phenomena like email records, chat logs, network logs, road maps, protein structures, or any of a large collection of problems that can be represented as a graph.

2.2.1.1 Scale-Free Graphs

Scale-free graphs are a naturally occurring graph class characterized by a distribution of vertex degrees that follow a power-law distribution. Scale-free graphs are common in social networks because as the number of vertices increases over time, new vertices preferentially attach to vertices that already have high degrees [57]. They tend to have a small diameter, a large variance in neighborhood size, and a 'global-neighborhood' characteristic where within a few hops all vertices are in the same neighborhood. These characteristics are often amalgamated together and termed the 'small world' property.

2.2.1.2 Mesh Graphs

Mesh graphs are a naturally occurring graph class characterized by a flat, regular degree distribution. Mesh graphs are often derived from physical phenomena like road networks, and so new vertices tend to extend the graph outwards [58]. They tend to have a large diameter, a small variance in neighborhood size and a 'local-neighborhood' characteristic where only a small subset of vertices are reachable within a few hops of any given vertex.

2.2.2 Synthetic Graphs

Real-world graphs offer realistic test inputs for graph algorithms. However, some domains lack sufficient data to support robust testing, especially regarding performance on irregular distributions. As a solution, synthetic graphs can be mathematically or imitatively generated to provide simple graphs for testing and evaluation.

2.2.2.1 Mathematically Generated Synthetic Graphs (Random Graphs)

Multiple mathematical functions can be used to generate synthetic graphs, typically by adding vertices and edges following some set of heuristics to reflect a given probability distribution. Notable generators include the Erdős-Rényi generator to create *Uniform Random Graphs* [59], the *Random Matrix* (RMAT) [60] and *Kronecker* [61] generator to generate scale-free graphs. Mathematically generated random graphs can be used to approximate real-world graphs but provide a sub-optimal substitute for the irregularities of real graphs because of the structural simplifications that occur in the generation process.

2.2.2.2 Imitatively Generated Synthetic Graphs (Imitative Graphs)

Many graph datasets are not releasable for research and development because of privacy and confidentiality reasons. Active research in privacy preservation and synthetic graph generation seeks to statistically summarize a real-world graph, and then use that summary as the input to a synthetic graph generator, so that the synthetic graph has will have the same topological properties as the original, without the confidential information. Imitative graph generation methods use input features like *Triangle Histograms, K-Hop Neighbor Histograms, Random Walk Histograms*

and *Number of Edges* to produce topologically similar graphs [62]. process is inherently lossy and has only been tested on very small graphs (tens of thousands of edges), so is not yet reliable enough for wide use.

2.2.3 Summary

The core graph topology classes discussed above (scale-free, mesh, mathematically generated, imitatively generated) each have inherent benefits and drawbacks for graph processing. We view these classes as representatives of the kinds of 'datasets' that are relevant to GraphAI . As such, a good GraphAI benchmark should cover each of these classes to enable a longitudinal view of expected performance.

2.3 Graph Representations

The representation and abstraction model of a graph fundamentally affect how it can be processed and operated on. The representation format of the input graph is a key contributor to the cost, effectiveness, and efficiency of GraphAI tasks operating on graphs. The tradeoff between memory efficiency (how much space a graph consumes) and lookup efficiency (how fast information can be accessed) is a core tension in graph processing. This section introduces key concepts of graph representation, including storage and abstractions as the primary levers used to balance memory consumption and compute efficiency.

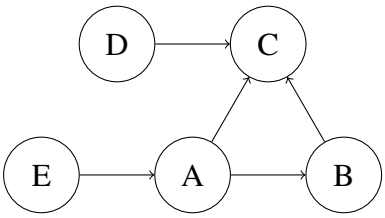


Figure 2.1: Example graph with five nodes and five edges

2.3.1 Graph Storage

A graph's storage format has a direct impact on the space it consumes in memory, the operations that can be performed on it, and the speed of those operations. In this section, we discuss the common ways a graph's topological information can be stored. In most cases, attribute information is stored separately from the graph and can be accessed via pointers or index lookups. The following examples show the common representations used to store the graph in figure 2.1.

2.3.1.1 Adjacency Lists

The adjacency list representation (Figure 2.2) uses a linked list or array. The first entry in each line is the source vertex and each subsequent entry indicates an edge to the listed vertex. Adjacency lists can represent directed or undirected graphs, and can represent weights using lists of tuples. a directed or undirected graph. Representing weights is possible using a list of tuples. Adjacency lists have a small memory footprint ($\mathcal{O}(|V| + |E|)$), but finding a particular edge is expensive – linear in the number of vertices ($\mathcal{O}(|V|)$) in the worst case.

$A, \quad B, \quad C$

$B, \quad C,$

$C,$

$D, \quad C,$

$E, \quad A,$

Figure 2.2: Adjacency List for graph in figure 2.1. Each line represents a vertex with the first in the list being the source and each subsequent vertex a destination of an edge.

2.3.1.2 Adjacency Matrices

An adjacency matrix (Figure 2.3) uses a $|V| \times |V|$ matrix (M) to represent the graph. A 1 at location $M_{i,j}$ indicates the existence of an edge from vertex i to vertex j. Adjacency matrices can represent directed or undirected graphs, and the value at position $M_{i,j}$ can be changed to represent a weighted edge. Adjacency matrices have a memory footprint of $\mathcal{O}(|V|^2)$ but a constant lookup time ($\mathcal{O}(1)$). One of their major drawbacks is the requirement to store all the 0s where edges do not exist, which wastes space.

2.3.1.3 Incidence Matrices

Incidence Matrices (Figure 2.4) use a $|E| \times |V|$ matrix (M) to represent the graph. An edge is represented with two values, namely a -1 at position $M_{i,j}$ to indicate the source and a 1 at position $M_{i,k}$ to indicate the destination. Here, i represents the index of the edge, and j and k represent the vertices that the edge connects. Incidence matrices are the most expressive representation of those listed here since they can encode *multi-graphs* and *hyper-graphs*. Weights

$$\begin{bmatrix} & A & B & C & D & E \\ A & 0 & 1 & 1 & 0 & 0 \\ B & 0 & 0 & 1 & 0 & 0 \\ C & 0 & 0 & 0 & 0 & 0 \\ D & 0 & 0 & 1 & 0 & 0 \\ E & 1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Figure 2.3: Adjacency Matrix for graph in figure 2.1 is a $|V| \times |V|$ matrix where a value of 1 indicates an edge and 0 is no edge.

can be expressed by modifying the numbers in the matrix. A multi-graph encoding requires inserting source values at $M_{a,j}$ and $M_{a,k}$ and destination values at $M_{b,j}$ and $M_{b,k}$, describing two directed edges going from vertex j to k. A hyper-edge encoding requires placing a source value at $M_{a,j}$ and destination values at $M_{a,k}$, $M_{a,l}$, and $M_{a,m}$, describing a single edge that connects the vertex j to k, l and m. The space complexity of an incidence matrix is $\mathcal{O}(|V| \times |E|)$, and so for sparse graphs the incidence matrix format may be more space-efficient than an adjacency matrix, but in general, it will consume significantly more memory in the typical case where $|E| \gg |V|$. Edge lookup using an incidence matrix is $\mathcal{O}(|E|)$, since it requires scanning the entire edge list for each vertex of interest, which is more expensive than the $\mathcal{O}(|V|)$ edge lookup using adjacency lists.

$$\begin{bmatrix} & e_0 & e_1 & e_2 & e_3 & e_4 \\ A & -1 & 0 & 0 & 1 & -1 \\ B & 1 & -1 & 0 & 0 & 0 \\ C & 0 & 1 & 1 & 0 & 1 \\ D & 0 & 0 & -1 & 0 & 0 \\ E & 0 & 0 & 0 & -1 & 1 \end{bmatrix}$$

Figure 2.4: Incidence Matrix for graph in figure 2.1 is a $|E| \times |V|$ matrix where a -1 is the source node and 1 is the destination of a given edge.

2.3.1.4 Compressed representations

To balance expressiveness, memory consumption, and computability, numerous compressed representations have been developed. The dominant choices are the Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC) formats (Figure 2.5). The CSR format uses three lists to compress the rows of a sparse (adjacency) matrix [63]. The CSC format reverses the format so that the columns are compressed. The space efficiency for CSR/CSC format is $\mathcal{O}(|V|\log(|M_{nonzero}|) + |M_{nonzero}|\log(|V|))$ for an adjacency matrix M, encoding $|V|$ vertices [63] and takes $\mathcal{O}(\log(|M_{nonzero}|))$ time to check the existence of an edge where $|M_{nonzero}|$ is the count of non zero entries in the matrix. More complicated representations exist that are capable of compressing the graph representations even further but they require more computational overhead for the encoding and decoding process [64].

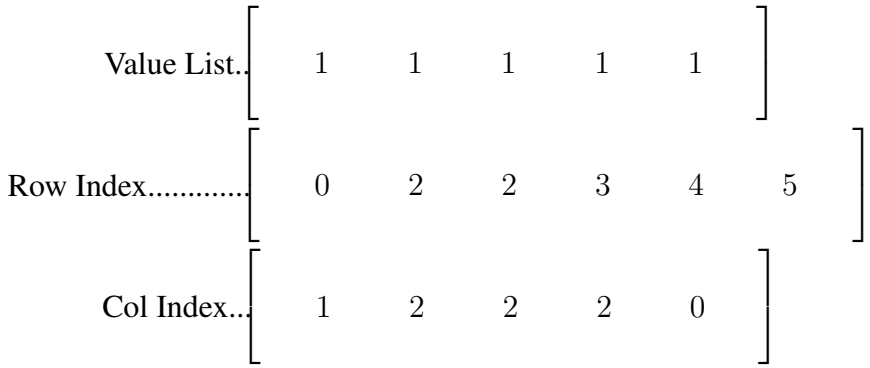


Figure 2.5: The Compressed Sparse Row (CSR) format uses 3 lists to represent the graph in figure 2.1.

2.3.2 Graph Abstractions

The abstract model of a graph refers to how its representation is simplified for storage and operations. There are two dominant graph abstraction paradigms, the *Canonical* and the *Linear Algebraic*.

2.3.2.1 Canonical

The canonical representation explicitly represents vertices and edges in memory. Operations on canonical graphs need to explicitly 'follow' the edges of a graph, traversing them in turn until target vertices are found. The canonical paradigm is more easily interpretable by humans and is generally used when explaining representations and operations.

To enable more powerful parallel processing methods and leverage hardware accelerators *vertex-centric*, *edge-centric*, *graph-centric*, and *data-centric* abstractions have been developed.

Vertex-Centric. The vertex is the central unit of data in this abstraction. The abstraction uses iterative operations on vertices, updating their states based on the properties of neighboring vertices [65]. Google's Pregel System [66] and Apache Girafe [67] follow the vertex-centric

approach.

Edge-Centric. T The edge is the central unit of data in this abstraction. Edges are updated based on the properties of the incident vertices [68]. The X-Stream [69] and GraphX [70] implement the edge-centric approach.

Graph-Centric. The graph itself is the central unit of data in this abstraction. This paradigm explicitly partitions the input graphs into subgraphs for processing [71]. ApacheGiraffe++ is a noted adopter of the graph-centric abstraction [72].

Data-Centric. The graph is the central unit of data in this abstraction. It is distinct from the graph-centric model in that rather than explicitly partitioning the graph, it implicitly segments the graph into 'active' and 'inactive' parts. That is, there is a single graph, but most of it is ignored at any given time. 'Active' nodes are added to a '*frontier*' that allows for more efficient computing by limiting the number of edges or vertices being processed at any time. The *Gunrock* framework introduced and champions the data-centric frontier abstraction [73]

These canonical abstractions all have tradeoffs. Vertex and edge-centric abstractions tend to inflict high communication overheads as each vertex and edge are treated as an individual unit. Graph-centric abstractions can struggle with optimally partitioning the graphs and load imbalance in the partitions but can reduce communication overheads by abstracting them to subgraphs. Data-centric abstractions using the frontier have shown significant improvements in execution time, but many implementations are unable to support certain classes of algorithms [74].

2.3.2.2 Linear-Algebraic

Linear-algebraic abstractions exploit the ability to represent graphs as sparse matrices using their incidence and adjacency matrices. The Combinatorial Basic Linear Algebra Subprograms (CombBLAS) [75, 76] and Graph Basic Linear Algebra Subprograms (GraphBLAS) [77, 78] are the two most popular linear algebraic graph abstractions. The linear algebraic abstraction exploits the latent coarse-grained parallelism within graphs and builds on advances in high-performance linear algebra processing to improve graph processing [75]. Both similarly identify that many graph operations can be composed of general and sparse matrix-vector multiplications so that actions like a single step in a BFS can be represented as a single linear-algebraic operation [79]. Linear algebraic representations are powerful and they offer a promising direction but they cannot support certain certain classes of algorithms, like those leveraging priority queues [75]. They can also be memory-intensive [80], and are less performant than implementations optimized for the hardware they run on [81].

2.4 Graph Primitive Operations

Primitives are the abstract building blocks that compose more complicated programs. For example, the *Select, Project* and *Join* primitives from relational database processing abstraction are combined in various orders to compose complicated queries and operations on databases. Having a set of common primitive operations is extremely helpful in building hardware-agnostic applications and gives hardware designers a set of operations to optimize to improve performance. However, there is no real agreement on what the building blocks of graph processing should be [82] since graph operations tend to be dependent on the abstraction model and implementation

used.

The simplicity of sequential graph operations has not driven the creation of any core primitive operations. The complexity of parallel operations on graphs has driven the development of several primitive families including the *Gather-Apply-Scatter* family derived from Map-Reduce [66] and the hierarchical collections of graph primitives built on work-efficient atomic and parallel processing primitives [64, 74, 83, 84]. Graph Manipulation Operation Primitives have inherited the *Create, Read, Update, Delete* primitives because of their proximity to databases [85], but are not utilized in most graph processing approaches. One of the most promising primitive families is Matrix Operations. Suites like the Combinatorial Basic Linear Algebra Subprograms (CombBLAS) and GraphBLAS use a linear-algebraic abstraction of the graph and matrix manipulations to carry out graph operations [75–79]. Matrix-based abstractions are particularly interesting because of their close relationship to other linear algebraic operations which have been the focus of years of hardware acceleration and performance optimization.

2.5 Architectural Constraints for Graph Processing: Locality

Modern general-purpose architectures optimize for general-purpose computing. Most computing tasks involve working on a few key pieces of data for a long time or accessing data that are close to each other because we store things together that we will need to retrieve together. Graph processing does not resemble general-purpose computing, and so the architectural optimizations to capitalize on accessing data that are close together and reusing important data have a diminished impact. Graphs lack locality.

2.5.1 Locality

Locality is a term in computer architecture that refers to the properties of data that enable it to be pre-emptively cached because it is nearby, and so is more likely to be needed than data that is far away (spatial locality) or is important and likely to be re-used soon (temporal locality).

Spatial Locality. We store related data together and the idea of optimizing for spatial locality is that when data is needed for processing, data stored near it in memory is also likely to be needed for processing soon, so proximal memory blocks can be pre-emptively fetched to the cache for quick access [56]. In figure 2.6, the vertices are stored in memory in *vertex-order*, that is, written to memory in the order they are created (irrespective of edges). This manner of storage typically leads to a situation in which the vertices that are close to each other in the graph topologically are not close to each other in memory. In other words, they lack spatial locality. The lack of spatial locality also means that only a small percentage of each cache line is used in each step as irrelevant information is retrieved just because it happens to be nearby. The irregular memory access problem is exacerbated in larger graphs, where it is more likely that the vertex list does not fit in the cache and the processor will have to access the main memory to retrieve the data [86], which is reflected empirically in a decreasing *instructions per miss* rate on larger graphs [58].

Temporal Locality. We tend to transition between related tasks, so data that is important for what we are doing now is likely to be important for what we do next. Temporal locality is the idea that when a piece of data is needed for processing, it will likely be needed for processing again soon, so it can be retained in the cache for quick access. To *exploit* the *temporal locality* systems can cache high-degree vertices since they are likely to be needed often [58]. There is

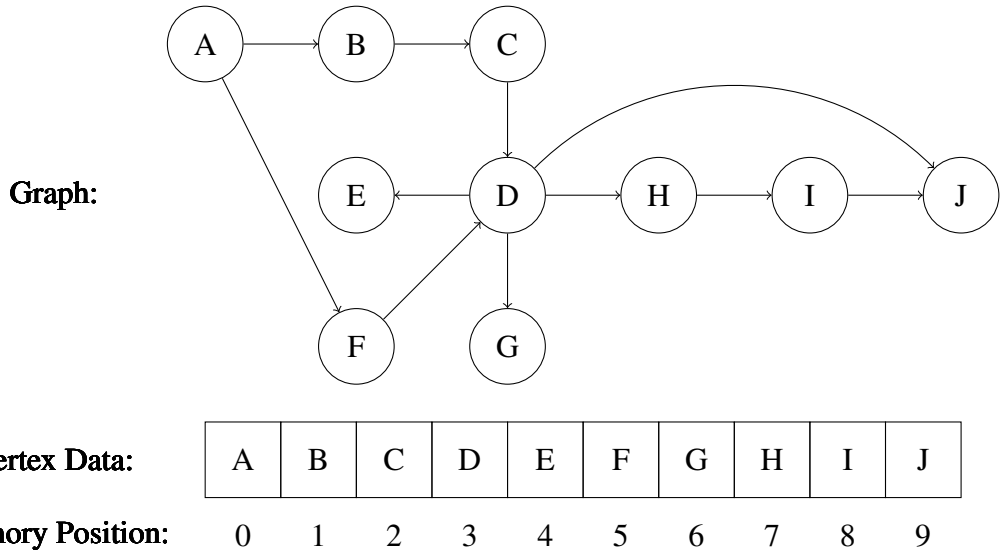


Figure 2.6: A simplified representation of a direct graph in vertex-order where each vertex is added to memory in the order it was created (here, alphabetically). Edges are not represented here.

some natural locality in *scale-free* graphs that can be readily exploited. Consider figure 2.6 again. While it is a very rudimentary sketch of a graph, vertex **D** has a much higher *degree* than the other vertices. It is likely to be visited multiple times during processing, so caching it will reduce the number of times it needs to be pulled from memory.

Attributed Graphs and Locality. The latency problem is exacerbated by attributed graphs because graph attributes are often stored separately in memory [87]. Figure 2.7 expands on the example from figure 2.6 by adding in vertex attributes. The attributes are stored at separate memory locations, with pointers from the topological vertex (memory positions $1 - 9$) to their respective attributes (memory positions $10 - 29$). Attributes consume more memory, so more memory bandwidth is required to load them. The heterogeneous data types being loaded in attributed graphs (like knowledge graphs) have been shown to increase the number of Dynamic Random Access Memory (DRAM) accesses and, hence, increase latency and memory bandwidth

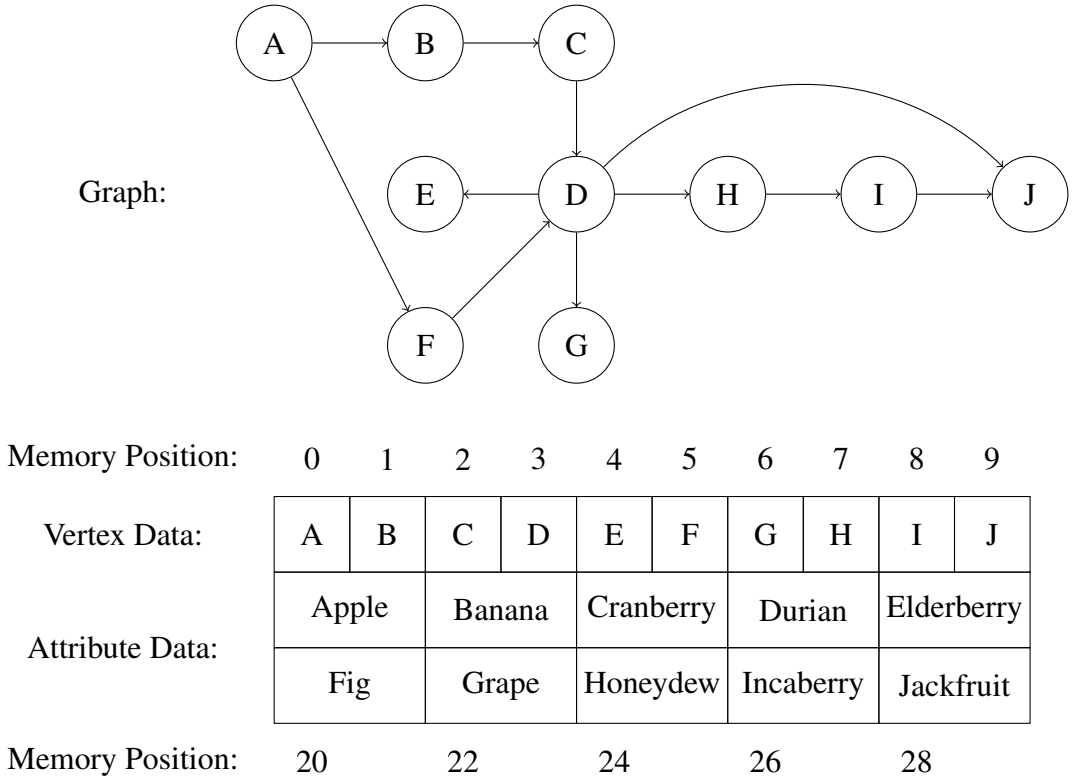


Figure 2.7: A simplified representation of a direct graph in vertex order where each vertex is added to memory in the order it was created with attributes stored separately. Pointers from topology to attributes not shown for clarity, but correspond to first letter of attribute.

pressure [88]. To illustrate why, again consider a BFS that is searching for the '**Grape**' vertex. Each time a vertex is dequeued, its attribute must be retrieved from memory to see if it is the target vertex. In the worst case, this is a memory access each time a vertex is checked, or, if reducing the latency of the comparison by pre-fetching the attribute along with the vertex, the cache block of 3 quickly becomes filled with the vertex plus its attribute, exacerbating the spatial locality problem.

Chapter 3

Literature Review

In this section, we motivate our methodological approach by reviewing benchmark design from an empirical point of view. First, we discuss the benefits and drawbacks of the two main types of benchmarks. Then, we review the GraphAI benchmark landscape to demonstrate critical gaps in the coverage of GraphAI tasks, datasets, and metrics. Finally, we survey workload analyses and summarize known GraphAI processing bottlenecks to emphasize that current benchmark approaches lack support for evaluating improvements on these bottlenecks. The result of the literature review is the design of our benchmark, which is described in chapter 4.

3.1 Benchmarking

Benchmarking is a process by which systems or components are compared according to some criteria, which is used to determine their relative performance [89]. Benchmarks like SPEC, LINPACK, and SATLIB have successfully been used to advance research in HPC, dynamic programming and satisfiability. [56, 90, 91].

Benchmarks can be *specification-based* meaning only input and output requirements are defined, *kit-based*, meaning substantive components are provided, or hybrid, meaning somewhere in between. Specification-based benchmarks provide formal definitions of tasks and users report their results back to them, whereas kit-based approaches often have some kind of instrumentation to measure the performance of a system. Because of their flexibility, specification-based

approaches like Graph500 tend to be more popular. Whether specification or kit-based, the two main approaches to benchmarking are *competitive* and *empirical* benchmarks.

3.1.1 Competitive Benchmarks

Competitive benchmarks are extremely popular in the AI community. They involve selecting a metric or metrics and 'racing' systems as black boxes to identify which one can maximize that metric. They are typically specification-based, and the only factors considered are the results generated and the time taken to generate them. For example, in computer vision, the Common Objects in Context (COCO) challenge for object detection uses average precision[1] [92]. Visual question answering (VQAv2) uses VQA accuracy[2] [93], and the LLM challenge ChatBot Arena[3] [94] uses 'user preference' as the metric.

Competitive benchmarks are useful for getting a quick overview of the current State of the art, but they have three major drawbacks. The first is that these provide a 'black box' view of the systems which makes it difficult to tell why one system performed better than another, or how to improve them. Competitive benchmarks show which systems are better but provide no information about why they are better, or how they can be improved [1, 95]. Second, is that this 'leaderboard' style of challenge encourages optimizing for the benchmark, rather than generally progressing the knowledge of the field [2]. The third drawback is that competitive benchmarks tend to emphasize effectiveness measures, ignoring efficiency altogether. The problem with overemphasizing effectiveness is that it is possible to miss the broader picture and reward a result that uses more resources to achieve a slightly better result than an efficient algorithm that uses fewer

[1] https://cocodataset.org/#detection-leaderboard
[2] https://visualqa.org/roe.html
[3] https://huggingface.co/spaces/lmsys/chatbot-arena-leaderboard

resources.

3.1.2 Empirical Benchmarks

Empirical benchmarks take the opposite approach to competitive benchmarks, instead focusing on identifying bottlenecks and providing a platform for further research [1, 2]. Empirical benchmarks follow either the workload analysis or hosted benchmark paradigms.

Workload Analyses. Workload Analyses are detailed white-box studies of the internal states of a system while undertaking a series of tasks on a collection of datasets. They focus on more detailed measurements of memory usage, network bandwidth usage and processor utilization. Workload analyses are invasive and require complete access to the system under test to instrument the implementation and profile the environment during execution. Reports are typically generated manually and focus on a single system or a comparison of a few systems. Although workload analyses provide a very detailed understanding of what is happening in a system they are work-intensive and do not scale well

Hosted Benchmarks Hosted benchmarks work by providing a shared environment in which implementations can be tested under fixed conditions. To ensure fairness, hosted benchmarks are typically maintained by third-party entities that accept submissions, run them and report the results. The hosted approach relies on shared problem formats and common primitive operations and constrains developers to implementations that can run on the provided environment. For example, the StarExec project for logic solvers [96] allows users to submit implementations, and have the results published to public leaderboards. Hosted benchmarks can accommodate comparisons across hardware, including across distributed and accelerated systems, but doing so

requires maintaining multiple concurrent hardware hosts, increasing cost while limiting participation.

3.1.3 Constrained Benchmarks

Constrained benchmarks are a promising alternative to competitive and hosted benchmarks. A constrained benchmark limits the time or resources available to competitors while requiring them to maximize their effectiveness metric. For example, the *efficientQA* efficient open domain question answering competition required competitors to maximize Question Answering Accuracy while adhering to strict memory budgets forcing trade-offs between the size of the models size of the reference corpora [97]. A constrained benchmarking approach shows how even without explicit access to the internal state of a system required for an empirical benchmark, we can still achieve a more thorough comparison than a simple competitive 'race' for effectiveness.

3.1.4 Summary

Current benchmarking approaches give us 3 paradigms. Competitive benchmarks are flexible, accessible computational footraces that tell us which system is the fastest or the most accurate, but not why. Empirical benchmarks are labor intensive and impose constraints on the systems being studied and while competitive benchmarks take a creative approach, they tend to focus on only one dimension during evaluation. The approach we propose in chapter 4 proposes combining the accessibility of competitive benchmarks with the informativeness of a formal empirical benchmark by leveraging the ideas from constraint-based benchmarks about how to stimulate interesting behavior in the system under test. The next section examines GraphAI specific

approaches to benchmarking, identifying how these paradigms are used and what their limitations are in practice.

3.2 Benchmarking GraphAI

Most of the influential GraphAI benchmarks are specification-based competitive benchmarks, however, it is common practice to provide specific real-world datasets, and reference implementations typical of kit-based benchmarks. From section 3.1 and the benchmarks this section surveys, we identify three core abstract components of a benchmark.

The three abstract components that form the benchmark specification are *Datasests*, *Tasks* and *Metrics*. We use these components as the focal points for the literature review of 23 prior benchmarks in graph analytics, graph databases (GDB) and graph neural networks (GNN) to identify gaps that a GraphAI benchmark should cover.

Focus Area	Benchmark	Number of Tasks	ETL	Search, Plan & Schedule	Structural Analysis	Reasoning	Graph Abstraction	Graph Prediction	Trustworthy AI
Graphs	Graph500 [98]	3	✓	✓					
	$PBBS_{1.0}$ [99]	7		✓					
	SNAP [100]	19	✓	✓	✓				
	$Graphalytics_{0.1}$ [101]	5	✓	✓	✓				
	$Graphalytics_{1.0}$ [102]	6		✓	✓				
	GAP [103]	5		✓	✓				
	SAGA-Bench [104]	5	✓	✓					
	GBBS [84, 105]	26		✓	✓				
	$PBBS_{2.0}$ [106]	6		✓					
	$Graphalytics_{1.2}$ [107]	6		✓	✓				
GDBs	SRBench [108]	3	✓	✓		✓			
	SocialGraph [109]	5	✓	✓					
	SocialBench [110]	6	✓	✓					
	TEKG [111]	4	✓	✓					
	BenchGraph [112, 113]	5	✓	✓					
GNNs	OGB [114]	4					✓	✓	
	PAGNNF† [115]	2						✓	
	GNNMark [20]	5						✓	
	Bridging-GNN† [116]	$2^{\ddagger}$						✓	
	LRGB [117]	5						✓	
	BGNN [118]	4						✓	
	CogDL [19]	8					✓	✓	
	OpenGSL [119]	1						✓	

Table 3.1: Coverage of Graph-Based AI tasks by Benchmark Suites and Workload Analyses (†) ‡=Inferred, not specified.

3.2.1 Task Coverage.

A benchmark task is a specific action that needs to be taken or a problem that needs to be solved, it specifies what the system must do for the benchmark run. We identify 23 graph benchmarks that satisfy at least one category from our GraphAI taxonomy . The reported tasks cluster around traditional graph tasks, GDB-specific tasks, or GNN prediction tasks depending on the focus area of the benchmark.

In table 3.1 benchmarks are shown grouped according to their focus area– traditional graph analytics (Graphs), Graph Databases (GDBs), or Graph Neural Networks (GNNs). For each benchmark surveyed, we list the total number of tasks it contains and whether at least one task covers each of the major problems in our GraphAI taxonomy. In addition to those categorizations, we add a column called ETL to reflect whether the benchmark measures the basic ETL operations that are critical for GDBs and dynamic graph processing. From the coverage of this column, it is clear that graph manipulations are often scoped out of traditional graph and GNN benchmarks.

We identify two key takeaways about the task coverage of current GraphAI benchmarks based on the information collected in table 3.1.

Tasks focus on specific operations, rather than general problems. In total, we identified 62 distinct GraphAI tasks by reviewing benchmark specifications, published results, and public code repositories, suggesting that a wide variety of task coverage exists in the literature, just not all in one benchmark. The benchmark with the broadest task coverage, GBBS [84], has 26 tasks, but the median coverage is 5 tasks, meaning most of the benchmarks tend to focus on a specific sub-area within a given GraphAI problem. Few benchmarks cover more than two of the seven GraphAI problem areas, and the areas cluster around 'like' problems. The focus on comparing

specific tasks drives the development of those tasks, which in turn drives the development of benchmarks to compare those tasks. Reviewing hardware accelerator designs emphasizes that systems are being built to focus on accelerating these 'narrow' benchmark kernel-tasks[4]., not broader real-world problems.

In contrast, we propose a system-agnostic with a problem-centric approach so that systems completing the benchmark will need to solve the problem, rather than just complete a narrowly specified kernel task in a shallow facsimile of the real world [56]. Part of the reason for this choice is to compose more complicated problems out of a collection of simple problems to better reflect the composite processing requirements expected in real-world settings [29].

ETL is under-represented in benchmarks given their importance in real-world GraphAI processing. Only nine of the surveyed benchmarks explicitly include tasks that require ETL, reflecting the problematic assumption that graphs are static, meaning the construction cost can be amortized over the life of the graph. As our overview of GraphAI problems in the real world from section from section 1.1 emphasizes, many problems are constrained by the dynamic nature of the graph. As a result, the limited coverage of graph manipulations makes it difficult to generalize their results to dynamic graph processing. Furthermore, only three benchmarks (SAGA-Bench [104], SRBench [108] and PAGNFF [115]) provide explicit support to dynamic graph tasks. The benchmarking framework we propose in chapter 4 explicitly includes ETL time in every benchmark problem. We create a benchmark that can accommodate static or dynamic processing approaches by tracking the *batch latency*, the length of time between when ETL begins on a datum and when it shows up in the results of a query of operation.

[4]In benchmarking, the smallest measurable 'thing' is an *atomic operation* (e.g. traversing an edge). Multiple atomic operations compose into a *kernel* (e.g. BFS), which can compose into *toy, synthetic* or *real* programs, which implement kernels to solve some higher level problem at varying levels of scale and realism [95]. See section **??** for further detail.

Focus Area	Benchmark	*Distinct Datasets*	*Max Edges*	*Attributed*	*Directed*	*Weighted*	*Graph Types*	*Data Origin*	*Input Format*
Graphs	Graph500 [98]	6	70T			(O)	SF	S_{UG}	AL
	$PBBS_{1.0}$ [99]	6	112M		✓	(O)	R,SF	S_{UG}	AL
	SNAP [100]	126	1.8B	✓	✓	✓	R,SF,M,MG,KG	R	AL
	$Graphalytics_{0.1}$ [101]	6	5B		✓	✓	R,SF	S_{UG}	AL
	$Graphalytics_{1.0}$ [102]	37	17B		✓	✓	R,SF	S_{PB}	AL
	GAP [103]	5	2.1B		✓	✓	R,SF,M	R,S_{PB}	AL
	SAGA-Bench [104]	5	500M		✓		R,SF,Dyn	R,S_{PB}	AL,HM
	GBBS [84, 105]	0	N/A	N/A	N/A	N/A	N/A	N/A	N/A
	$PBBS_{2.0}$ [106]	6	112M		✓	(O)	R,SF	S_{UG}	AL
	$Graphalytics_{1.2}$ [107]	37	17B		✓	✓	R,SF	R,S_{PB}	AL
GDBs	SRBench [108]	3	1.76B	✓	✓		SF,KG, Dyn	R	RDF
	SocialGraph [109]	5	161M		✓		SF,KG	S_{UG}	RDF
	SocialBench [110]	4	20B		✓		SF,KG	S_{UG}	RDF
	TEKG [111]	1	1T		✓		SF,KG	S_{UG}	RDF
	BenchGraph [112, 113]	9	52M		✓		SF,KG	S_{PB}	Cypher
GNNs	OGB [114]	15	1.6B	✓	✓		SF,KG	R	SFF
	PAGNNF† [115]	5	44.4K	✓	✓		SF,M,Dyn	R	SFF
	GNNMark [20]	12	1.9M	✓	✓		SF,M	R	SFF
	Bridging-GNN† [116]	8	21K	✓	✓		SF,M	R	SFF
	LRGB [117]	8	2.3M	✓	✓		SF,M	R,S_{PB}	SFF
	BGNN [118]	5	2693K	✓	✓		SF,M	R,S_{PB}	SFF
	CogDL [19]	81	132M	✓	✓		SF,M	R	SFF
	OpenGSL [119]	10	2.24M	✓	✓		SF,M		✓

Table 3.2: Coverage of Graph-Based AI Datasets by Benchmark Suites and Workload Analyses (†)

3.2.2 Dataset Coverage.

Datasets are a core component of benchmarks. They are the 'thing', or 'problem' that a system must conduct a task on. A collection of standard datasets makes the results between different systems comparable. SinceGraphAI performance is highly dependent on the size and shape of the input graph, good performance measurements require broad coverage of graph shapes from the domain of interest. We surveyed the datasets used in 23 graph benchmarks finding that the median number of input graphs is 6 per benchmark and that there is a general focus on real-world scale-free graphs. The benchmarks are grouped by their focus area (graph analytics, GDB, GNNs) in table 3.2, and the column entries indicate whether each benchmark satisfies a given graph property with at least one of the datasets it contains. We identify three areas of concern in dataset coverage for GraphAI benchmarks.

Heterogeneous graphs are not well covered. Real-world GraphAI problems require processing heterogeneous graphs consistent with knowledge graphs [29]. Of the benchmarks surveyed, heterogeneous graphs were sparsely covered in all focus areas except GDBs. Traditional Graph Analysis and GNNs both focus on topological features, leading to a scarcity of complex attributed graphs for benchmark datasets. Sourcing and if necessary, creating complex heterogeneous attributed graphs is an important focus for developing a GraphAI benchmark.

Benchmarks datasets vary in domain coverage rather than graph topology. The median number of datasets used by benchmarks is 6, with most selections being loosely justified and not explicitly tied to any domain. The benchmarks we surveyed tend to contain publicly available graphs about graph-native problems like mapping citation networks, analyzing friend networks on social media, and email events, though between SNAP [11] and the Network Data Reposi-

tory [120] there are numerous graphs available from health, physical sciences, geospatial, and many other domains. Only the GAP [103] benchmark explicitly chooses its datasets to represent mesh, scale-free and uniform random topologies to enable performance measurement across varied input shapes. For our approach, we will build on the GAP methodology and aim for datasets from at least the scale-free, random and mesh structures for every problem in the framework to generate data about the impact of varying data shapes on tasks.

Dynamic graph datasets are very uncommon. Dynamic graphs have spatio-temporal characteristics that are not common in static graphs. Edges and vertices can be added or removed at any point, and the graph must conduct ETL concurrently with completing GraphAI tasks. These dynamic approaches focus attention on providing results within a limited timeframe. The lack of dynamic graph datasets means that there is no clear drive to build and evaluate dynamic GraphAI systems, and so the development of time and resource-constrained GraphAI approaches slows. Because of the lack of available datasets, research on dynamic graphs needs to artificially transform static graphs into dynamic ones through random sampling However, many of the same drawbacks of synthetic graph generation apply to an artificial sampling of static graphs. For example, random sampling does not replace the natural pattern of vertex and edge creation, modification, and deletion that is inherent to dynamic graphs. Randomly selecting vertices and edges to add to the graph is unlikely to reflect how they would occur in the real world. The GraphAI benchmark framework is actively searching for dynamic graph datasets and intends to integrate them into every GraphAI problem.

Focus Area	Benchmark	*Effectiveness*	*ETL-Time*	*Exec-Time*	*Throughput*	*Compute*	*Memory*	*Network*	*Power*
Graphs	Graph500 [98]		✓	✓	✓				
	$PBBS_{1.0}$ [99]			✓					
	SNAP [100]			✓					
	$Graphalytics_{0.1}$ [101]			✓	✓				
	$Graphalytics_{1.0}$ [102]			✓	✓				
	GAP [103]		✓	✓					
	SAGA-Bench [104]		✓	✓	✓		✓		
	GBBS [84, 105]			✓					
	$PBBS_{2.0}$ [106]			✓					
	$Graphalytics_{1.2}$ [107]			✓	✓				
GDBs	SRBench [108]	✓	✓	✓	✓				
	SocialGraph [109]			✓					
	SocialBench [110]			✓					
	TEKG [111]			✓					
	BenchGraph [112, 113]			✓			✓		
GNNs	OGB [114]	✓					✓		
	PAGNNF† [115]		✓	✓		✓	✓		
	GNNMark [20]			✓			✓		
	Bridging-GNN† [116]			✓			✓		
	LRGB [117]	✓							
	BGNN [118]	✓		✓	✓				
	CogDL [19]	✓		✓					
	OpenGSL [119]	✓		✓			✓		

Table 3.3: Coverage of Graph-Based AI Metrics by Benchmark Suites and Workload Analyses (†)

3.2.3 Metric Coverage.

Metrics are the summaries of a system's efficiency, effectiveness and cost that we generate by observing how a system solves a task on a given dataset. The metrics generated by GraphAI benchmarks enable comparison of the relative effectiveness, efficiency and cost of GraphAI systems. We survey the coverage metrics of 23 graph benchmarks from published results and public leaderboards in table 3.3. We group benchmarks by their focus areas (graph analytics, GDBs and GNNs), and indicate whether each benchmark reports a given metric in at least one of its workloads.

We identify four areas of concern in the coverage of metrics by existing benchmarks.

Collecting detailed metrics is invasive and impractical. To collect interesting and useful metrics about the internal state of the GraphAI system (compute, memory, network and power), the system under test needs to be under the control of the benchmark owner so that the code can be correctly instrumented and the underlying system sampled throughout execution. As a result, these 'internal' metrics are only observed in detailed workload analysis studies where the researcher has access beyond the inputs and outputs to the system and what can be externally observed. In the remainder of cases, execution time is typically the only viable metric. The Input/Output model for benchmarking is flexible but does not facilitate the discovery of bottlenecks, which is critical for understanding which parts of the system should be improved. In the GraphAI benchmark framework, we use an empirical approach design with constraint theory to balance the benefit of a detailed workload analysis with the flexibility offered by a simple Input/Output design.

Throughput measures can be misleading. Throughput metrics are generally estimates of

traversals per second, which is computed by counting the edges in the input graph, estimating the traversals required by a single iteration of the algorithm and multiplying that by the number of iterations to get the total edge traversals, which is then divided by the execution time to measure traversals per second [121, 122]. The drawback of this estimation strategy is that it does a poor job of algorithms that aggregate or otherwise change the structure of the graph at each iteration. In the GraphAI benchmarking framework we focus on execution time as a downstream indicator of throughput and the interaction of throughput with the other aspects of the system.

Effectiveness is neglected Benchmarks measure performance by efficiency metrics or effectiveness metrics, but not both. Few of the surveyed benchmarks reported effectiveness metrics, likely because many 'traditional' graph problems are deterministic and were either correct or not. On the other hand, benchmarks for newer problems like Long Range Graph Learning [117] often only include effectiveness metrics, ignoring time and space efficiency until the problem can be effectively solved. The GraphAI framework takes a more nuanced view of correctness, linked to defining problems in general terms rather than as specific tasks and to constraint-based benchmarking. We intend to develop problems with degrees of correctness to encourage work on iterative, incremental and approximate solutions that may offer a trade-off between effectiveness for efficiency, giving a benchmark user options to experiment with their constraints to find the solution that is best for them.

Cost measurement requires system access. Cost measurements estimate how expensive a particular solution is and form the third dimension of metrics. However, none of the surveyed benchmarks make estimates of cost, and few acknowledge its proxies like power consumption. A complete model of system constraints needs to trade off efficiency, effectiveness and cost. For the time being the GraphAI benchmark framework is focusing on effectiveness and efficiency

because of the lack of methods for non-invasive cost estimation.

3.3 GraphAI Bottlenecks

A *bottleneck*, or *system constraint* is any resource whose capacity is less than or equal to the demand placed on it [123]. Design tasks that will discriminate GraphAI system performance requires understanding the common bottlenecks in GraphAI systems. This section summarizes the primary GraphAI bottlenecks that are commonly identified in GraphAI workload analyses– *memory, compute* and *communication* bottlenecks.

3.3.1 Memory Bottlenecks

Memory bottlenecks are when the maximum capacity of a system is limited by the speed of reading from and writing to memory. Memory bottlenecks occur when a system is unable to keep a processor fully occupied because the data required for computation cannot be moved from memory to the processor quickly enough [56]. They occur in graph processing because of the lack of locality and consequent underutilization of the cache hierarchy described in section 2.5 Subproblems contributing to memory bottlenecks are memory bandwidth utilization and memory latency Memory bandwidth defines how much can concurrently be moved to and from memory. It becomes the bottleneck when there is poor cache utilization [124], competing parallel processes [125, 126] or irregular memory access patterns [127–129] causing the processor to stall because not enough data is getting to it [82]. Memory latency defines how much time it takes to retrieve required data from memory, and exacerbates the under-utilization of memory bandwidth [115, 130]. Latency is problematic in GraphAI applications because the irregular data

accesses coming from the inherent lack of spatial and temporal locality (on average) [131, 132] cause cache misses [58, 128, 129] and inefficient cache usage even when it is hit [127, 132, 133], leading to slow main memory accesses so inefficient that even optimized pipelined architectures can't 'hide' the latency [134]. The GraphAI benchmark framework will use input data as the mechanism to trigger memory bottlenecks. Graphs with high vertex counts and high (but regularly distributed) degrees will induce the pointer-chasing behavior that forces memory bottlenecks to manifest.

3.3.2 Computation Bottlenecks

Computation bottlenecks occur when the system can serve data to the processor faster than operations can be performed on them. Optimizations for general-purpose architectures fail to deliver many benefits because of the irregular data-dependent executions in graph processing [135]. For example, branch misprediction is common [127], and distributed workloads frequently become unbalanced [127, 136]. Looking at GNNs, they have hybrid computing patterns that combine irregular graph processing and regular dense neural network processing. As a result, they suffer from both memory and computation bottlenecks, and so are difficult to implement efficiently on general-purpose architectures [137]. To mitigate the impacts, GNNs follow some of the parallel graph processing approaches by breaking algorithms into collections of the gather, apply and scatter primitives so that they can take advantage of distributed and accelerated computing. For the GraphAI benchmark we pursue an Input/Output benchmark design so that a benchmark user can bring their own system to test, trailing different hardware accelerators and system configurations to try and elevate the computation bottleneck.

3.3.3 Communication Bottlenecks

Communication bottlenecks occur in part of the system limiting the capacity is the network interface. They are caused by transferring large amounts of data, and by communication-intensive algorithms that require a lot of network traffic to stay synchronized. For graph processing, communication bottlenecks occur in parallel computing settings or when an external hardware accelerator is in use. These are caused by separation, movement, and synchronization overheads. Parallel processing requires the data to be divided up between workers. Partitioning graphs is a known NP-complete problem and the time taken to partition graphs for processing has a significant impact on overall efficiency [65]. When partitioned, the graph must then be copied to a distributed worker node (or an accelerator) and the time taken to copy the data for processing degrades efficiency [138, 139]. Synchronizing parallel worker nodes is challenging when the algorithm needs to access all vertices each iteration [82] or when the partitioning is sub-optimal [140]. distributed parallel approaches incur both of these penalties, resulting in severe communication bottlenecks [58, 84]. As for computation bottlenecks, the GraphAI benchmark uses a non-invasive Input/Output approach to give benchmark users the freedom to experiment with different architectures to explore the impact on the communication, computation and memory bottlenecks.

Chapter 4

The Graph Artificial Intelligence Benchmarking Framework

In this section, we present our GraphAI benchmarking methodology that takes a hybrid competitive and empirical approach and addresses the lack of domain-specific task, dataset, and metrics coverage that we identified in the previous chapter.

4.1 Motivation

In reviewing prior work in competitive benchmarking and workload analysis (section 3.1) we conclude that purely competitive benchmarks from the Input/Output paradigm lack descriptive power and that empirical benchmarks from the workload analyses paradigm are too invasive for general-purpose use. Utility must balance explainability, and our hybrid approach borrows ideas from constrained benchmarks, creating external levers to facilitate a more detailed 'black-box' approach to understanding the internal state of the system.

The review of GraphAI benchmarking (section 3.2) concludes that the existing GraphAI task, dataset and metric coverage is narrow, neglects dynamic processing of heterogeneous graphs, and fails to appropriately evaluate approximate or incremental solutions. Rather than taking a system-led approach, we take a problem-centric approach. We focus on the GraphAI problem categories (see table 1.1), derive relevant tasks within those categories from a study of I&S domain requirements (see table: 1.2), and design a benchmark framework that defines the tasks, datasets and metrics for evaluating across the breadth of GraphAI implementations and environments. The

problem-centric approach offers the flexibility to vary any benchmark component from *algorithms*, *implementations* and the underlying *hardware* within a unified framework, while monitoring the changes in efficiency and effectiveness.

4.2 Background

When designing a benchmark, careful consideration must be given to the underlying design principles and which components to include. This section briefly extends the literature review from chapter 3 by reviewing specific work in benchmark design and construction, to identify guiding principles for the construction of our benchmark.

4.2.1 Benchmark Design Principles

Benchmark design must consider the purpose of the benchmark, general principles of benchmark design and the granularity of the problem [141]. The *purpose* of the benchmark defines how a designer scopes the benchmark and applies the four core principles of benchmark design (*relevance, fairness, reproduciblity* and *verifiability*) [89, 142]. The purpose of the benchmark also drives its level of *granularity*. From most to least granular, benchmarks of *atomic operations, kernels, toy programs, synthetic programs and real programs* provide different information about the systems under test and the metrics that describe them [56]. Most contemporary benchmarks focus on independent kernel operations. That focus can be problematic because making inferences from a multitude of small kernels alone does not give a realistic understanding of holistic system performance 'in the real world' [56].

The GraphAI benchmark design follows the principles of benchmark design and extends

beyond the popular kernel-only benchmarks to balance understanding the performance of individual components with the performance of the system as a whole.

4.2.2 Components of a Benchmark

Existing benchmarks have two major components, the abstract *benchmark specification* and the concrete *system under test* (SUT). These two components interact, and those interactions are what the benchmark measures.

4.2.2.1 Benchmark Specification

The vertical swim lanes represent the benchmark specification components in figure 4.1. *Datasets* are drawn from a domain of interest, *tasks* are drawn from a relevant problem space, and *metrics* are tailored to reflect the specific evaluation criteria. More specifically:

Datasets. are the challenges that a benchmark needs to 'solve'. Figure 4.1 shows that source domains provide many benchmarks that become input to a SUT. The SUT receives the dataset (the 'challenge') and auxiliary information like expected answers to facilitate evaluation.

Tasks. are the specific processes that a benchmark needs to complete as part of the evaluation process. They are derived from the problem space (figure 4.1) and define a specific problem to be solved. A task is a blueprint for an implementation Together, a dataset and a task comprise a *workload*.

Metrics. define, collect and compute the observable interactions of a SUT and a workload. Figure 4.1 shows how the metrics are defined abstractly as evaluation criteria, collected from the outputs of the SUT as measurements and then compared to develop a relative performance

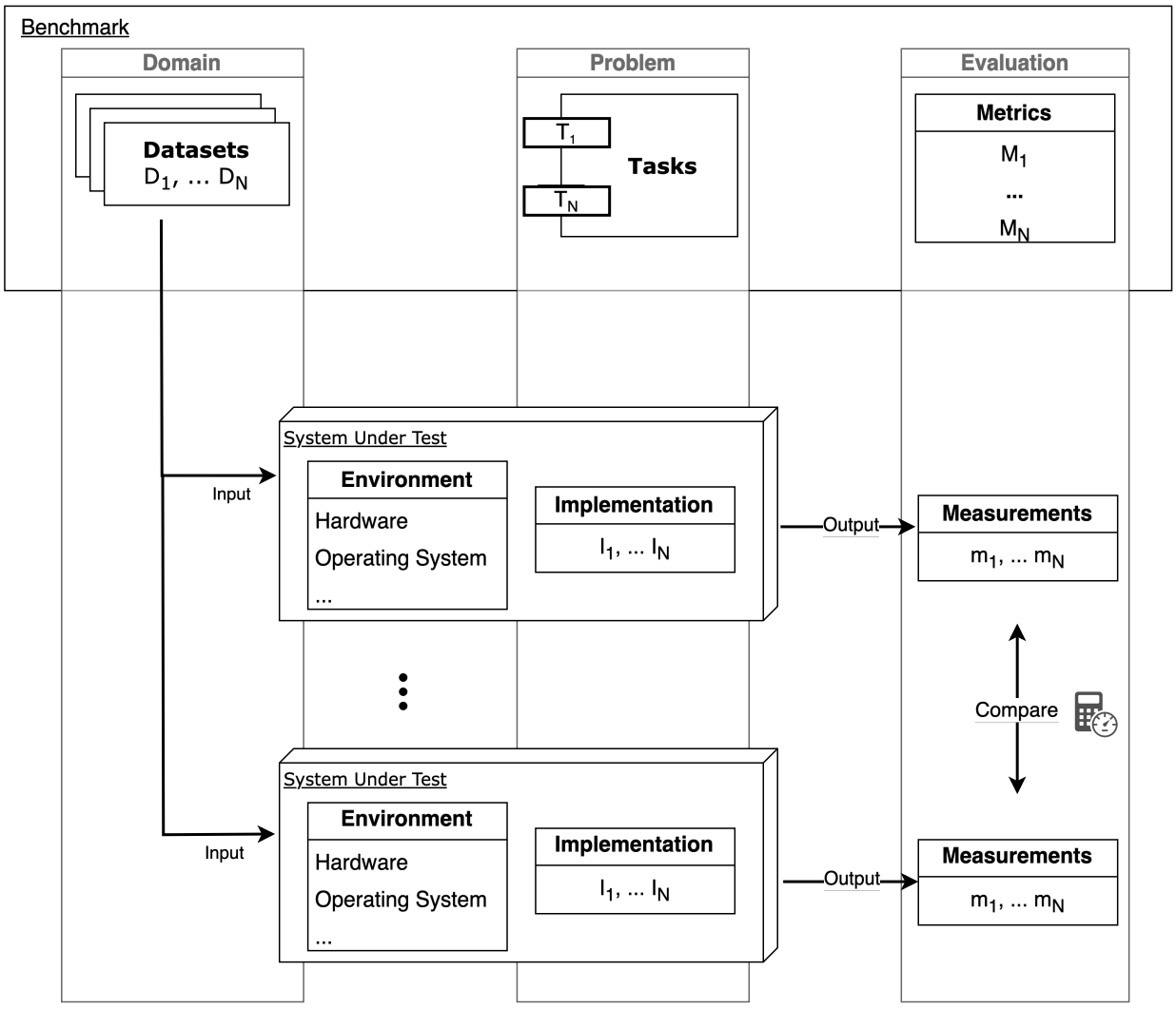


Figure 4.1: The basic components of a generic benchmark system

model between systems. The most common metrics describe efficiency, effectiveness and cost of running a given workload on a SUT.

4.2.2.2 System Under Test

The systems under test (SUT) by the benchmark are comprised of the *environment* which consists of the *hardware* and underlying *software stack* and the *implementation* which are the specific programs written to execute the workloads specified by the abstract dimensions of the

benchmark. Figure 4.1 shows how the same benchmark specification can have many SUTs complete the same tasks on the same datasets to generate the same metrics for comparison.

Hardware. is the environment component that refers to all the physical components of the SUT, including host machines, hardware accelerators or distributed systems. Hardware is the most commonly studied component of the SUT in benchmarks and workload analyses and is a popular target for acceleration. Being able to identify under what circumstances the use of a hardware accelerator is justified is a useful result of this benchmarking framework. Figure 4.1 shows that the SUT is distinct from the domain datasets and problem tasks, but is essential to bridging them

Software Stack. is the environment component that refers to all the software components that are not directly part of the implementation, including the operating system and enabling tools like a database where the focus of the evaluation is not on the database itself.

Implementation. The implementation is how the SUT approaches a given task – the instantiation of the blueprint (reflected in its positioning in figure **??**components aligned with the problem dimension that specifies the abstract tasks). Implementations consist of the abstraction model, data structures, algorithms, and programming language used to solve a given GraphAI problem."

The SUT is ultimately the target of the evaluation and that is defined by the benchmark specification. That is why in figure 4.1 they are shown as blocks that span (but are distinct from) the three specification dimensions. The distinction between the specification and the SUT is clouded in kit-based benchmarks that provide reference implementations as baselines of performance, or host-based benchmarks that expect all implementations to run on provided hardware.

Varied Component		Dependent Metric		
Dimension	Components	Effectiveness	Efficiency	Cost
Specification	Datasets	✓	✓	✓
	Tasks	✓	✓	✓
	Metrics			
System under Test	Hardware		✓	✓
	Software		✓	✓
	Implementation	✓	✓	✓

Table 4.1: Summary of dependencies between benchmark components and evaluation metrics

4.2.2.3 Experimental Design

Experimentation using benchmark suites involved modifying the benchmark specification or changing the SUT. Table 4.1 summarizes what metrics are expected to change when one benchmark component from figure 4.1 is varied, assuming all others are fixed. For example, as the hardware component of the SUT is varied, assuming all other factors are fixed, we expect to see a change in the efficiency and cost metrics, but not in effectiveness.

4.3 Design

Our design of the GraphAI benchmarking framework draws jointly from the established methods of benchmark design, and methods for the empirical studies of algorithms [1, 2]. The prior work in benchmark design guides *what* components to include, namely the *datasets, tasks, metrics* and *system under test*. The prior work in the empirical analysis of algorithms determines *how* we determine the composition of those components to maximize information gain.

4.3.1 Empirical Benchmark Design

The empirical approach to computer science applies the scientific method to design and execute controlled experiments on systems [2]. We use the empirical approach as part of the GraphAI benchmark design for balance, grounding and formality. Our empirically inspired approach balances the utility of competitive benchmarks with the detail of a formal workload analysis. Empirical studies tend to focus on typical problems, and real-world distributions of data to characterize what a typical class of problems that a system or algorithm must face are [2], so it supports us grounding the benchmark in the problems we identified in section 1.1 that are not covered by the current suite of available benchmarks (see: section 3.2). Most importantly, it provides a formal structure that improves the transparency and reproducibility of the benchmark development process.

The methodology we build from is John Hooker's framework for the empirical analysis of algorithms, summarized in figure 4.2. His method identifies four key components, the *problem*, the *approach*, the *phenomena* and the *apparatus*. The *problem* here studies what it actually is that we are trying to do (achieve some outcome in the domain and problem space). The *approach* here studies the dataset and task features we can use to stimulate the problem (identify datasets, tasks). The *phenomena* studies the interactions of an approach with a problem (identify suitable metrics and where to find them). The *apparatus* are the tools and techniques we use to investigate, measure or otherwise study those interactions between the phenomena and the problem(get datasets, implement tasks, instrument system, monitor execution etc).

Identifying and Characterizing a Relevant Problem. Understand the problem that you are trying to solve, including typical tasks, datasets and common challenges. For GraphAI it is a

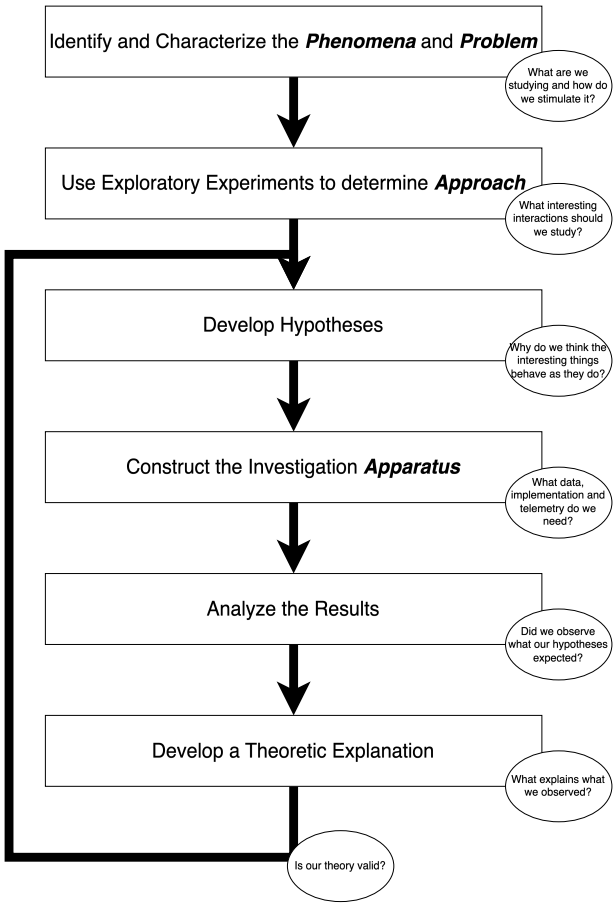


Figure 4.2: Hooker's methodology for an empirical science of algorithms [1, 2]

scoping phase, selecting on of the problems identified in the problem survey (see: section 1.2)

Identifying and Characterizing the Phenomena of Interest. What 'interesting things' are we expecting to observe, and why will measuring that help us understand the underlying problem? For GraphAI interesting things could be interactions between datasets and execution time, or general curiosities about how a system will respond in a given circumstance.

Conduct Exploratory Experimentation. Exploratory experimentation gathers the foundational understanding of the bottlenecks, friction points and curiosities of the problem required to design meaningful experiments, and how to stimulate them.

Develop Hypotheses. The hypotheses focus the analysis and inform data collection by

specifying what phenomena we expect to observe given a problem and an approach under study.

Construct the Investigation Apparatus. The investigation apparatus determines how data is collected, and the degree of freedom given to the systems under test. Importantly, this step collects or generates the test data. Hooker prefers synthetic data generation because generators can be parameterized to test specific data-dependent hypotheses, generating problems from 'easy' to 'hard'.

Analyze the Results, Developing a Theoretic Explanation and Iterating. The remaining three steps are more strongly in the domain of the person using the benchmark than the benchmark itself. The benchmark should provide a report of the efficiency, effectiveness and cost scores associated with the workload(s) completed, but further work is beyond the scope of the benchmark. This section is complete when the metrics have been reported to the user of the benchmark.

Once these steps have been completed, the benchmark for a specific problem is ready for use. The steps must be repeated separately for any additional problem that is added to the benchmark.

4.4 Abstract Framework

The GraphAI benchmark framework is presented below and summarized in figure 4.3. The framework is the overarching term given to the collection of GraphAI problems that comprise the benchmark. Each problem is separately designed and implemented with its own task definitions and datasets. The approaches represent an implementation on a SUT that is attempting the problem.

This section will present each of the steps in our proposed benchmark development methodology that is summarized in figure 4.4. We focus our descriptions on how the questions in the bottom quarter of each step generate the expected outputs from each step, and how the transition questions in the grey circles should inform the designer's thinking. The diagram also shows which components of the benchmark framework the outputs of each step directly contribute to.

We present the key methodological considerations at each step to develop *problem* to be added to the benchmarks and make some general observations about the requirement for and design of the observation apparatus. Chapter 5 demonstrates a worked example of how the methodology is implemented for a specific problem.

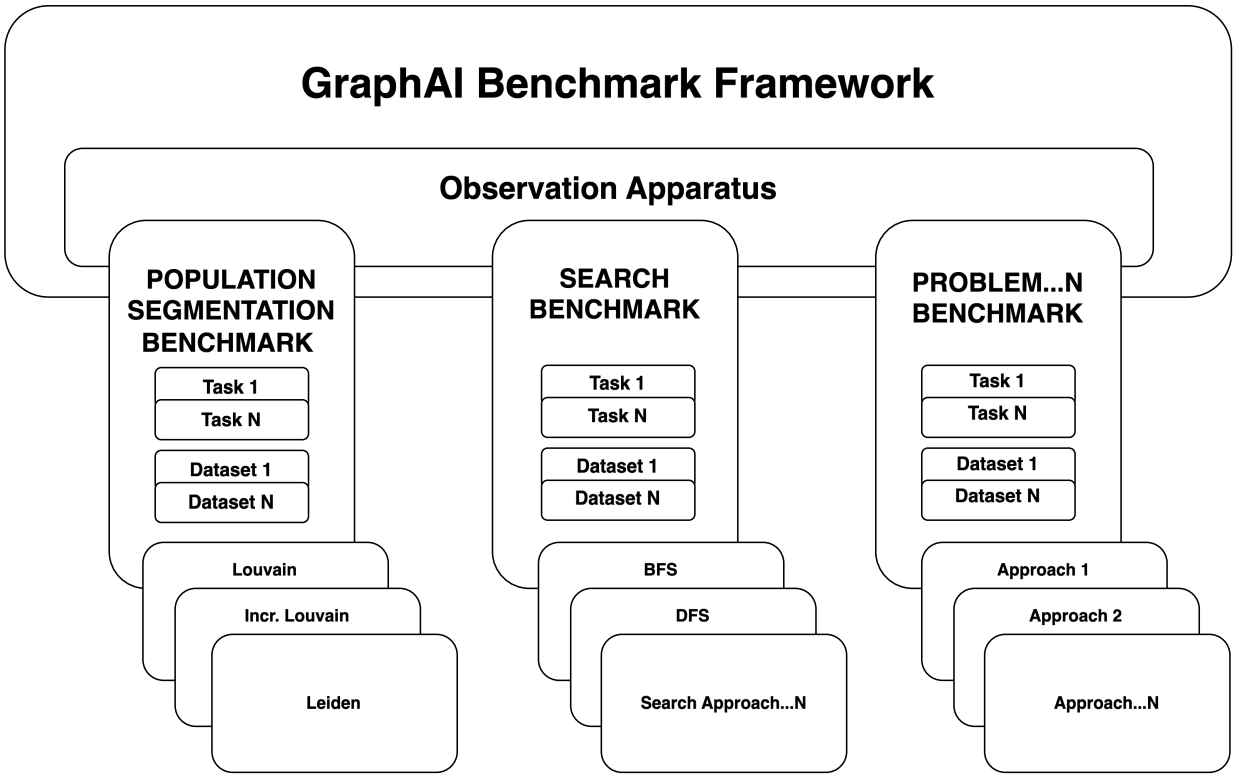


Figure 4.3: The GraphAI Benchmark Framework is a collection of empirically developed problems that share a common observation apparatus for timing and evaluation.

Identifying the Use Case The first step in creating the benchmark step is to identify the

underlying domain for the benchmark and select several problem use cases. The use case may be driven by necessity, or the target of exploratory research. The domain should be from the real world, but provided there are open problems, and some representative datasets are available, most problems can for formalized into a use-case.

Identifying and Characterizing a Relevant Problem. Figure 4.3 shows that problems are the combination of the task and the dataset. Characterizing the problem is about identifying what it is that we want to test, and identifying the tasks and datasets typical of the domain with a background study. Tasks should be defined as general goals and challenges rather than specific *kernels*. The generality subtly encourages formulating real problems into tasks instead of abstracting them to a kernel lacking context and also encourages innovative approaches. The intent is to shift the focus from racing optimized implementations against each other to figuring out how to best trade off time and accuracy For example in GraphAI , a problem definition might be *Given the input X, identify who the immediate associates of a person with name Y, provide the output as a list of vertex IDs* rather than *BFS to node X*, *search for X in a GDB* etc. The problem-based approach still allows for the comparison of like-approaches (e.g. comparing the same BFS algorithm on two different SUTs), by freezing all other components of the benchmark and just varying the SUT hardware.

Identifying and Characterizing the Phenomena of Interest. The goal of characterizing the phenomena is to identify what it is that we want to understand about the problems, and what metrics we need to collect towards understanding it. The phenomena of interest is the trade-off between the efficiency, effectiveness and cost of GraphAI systems. Thus, the metrics that should be selected for the benchmark should characterize the trade-off between those aspects of performance. While not shown explicitly in figure 4.3, the phenomena are generated by the interactions

of the approaches, datasets and tasks within a given problem and collected by the observation For example, in most GraphAI tasks we expect *ETL Time, Execution Time, Batch Latency*[1] and *Cost*. For each specific problem, the phenomena must be tailored so that the assessed 'goodness' of an output is meaningful.

Conduct Exploratory Experimentation. Exploratory experimentation provides the initial clues about where the bottlenecks, friction points and curiosities are that are worthy of further study. We are trying to understand how the phenomena relate to the problems and develop theories that we can explore formally later in the design process. Experimentation can be direct, using the problems and datasets identified to test out some simple theories about expected performance. It can also be indirect, relying on a synthesis of prior experimental work to develop an understanding of the expected phenomena arising from a given approach to a particular problem. In GraphAI , reviews of workload analyses, the findings of prior benchmarks and the designs of acceleration techniques offer valuable insight into what approaches are going to interact with what data to produce interesting phenomena.

Deveop Hypotheses. Developing hypotheses is a scientific approach to quantifying the behavior we expect to observe in the system and designing circumstances that will stimulate those behaviors. Hypotheses are developed in this methodology by using the knowledge gathered during exploratory experimentation. Hypotheses can be anything, but it can be helpful to derive them by applying the three constraint levers of time (efficiency), resources (cost) and accuracy (effectiveness) to develop them. These levers provide a mechanism to identify the trade-offs in GraphAI between efficiency, effectiveness and cost by constraining one (or more) and observing

[1] The time between when a vertex or edge is added to a graph and when it appears in the result of a workload [104]

Constraint	**Dependent Metric**		
Components	Effectiveness	Efficiency	Cost
None (Baseline)	—	—	—
Time	↓	—	↓
Accuracy	—	↑	↓
Cost	↓	↓	—

Table 4.2: Summary of expected metric behaviors (↑ Improve, — Stable, ↓ Degrade) when a benchmark component is constrained

changes in the phenomena under observation. Table 4.2 summarizes the behavior we expect to stimulate from an SUT by constraining time, accuracy or cost across any GraphAI problem, and are a resource from problem designers to build on.

Hypotheses should be formally specified in the form:

"When [lever] is constrained for [task] on [dataset] we expect [change] in [phenomena]"

Construct the Investigation Apparatus. Constructing the investigation apparatus has three key steps. First is collecting or generating datasets that will stress each hypothesis to stimulate the phenomena we intend to observe. The second is defining the metrics of interest and developing a mechanism to collect the data needed to generate them. The third step is integrating the SUT into the apparatus In an Input/Output paradigm, this will just be some API configurations, but for a hosted solution may involve test compilations etc. Specific to the GraphAI benchmark is a need to trade-off between effectiveness and efficiency that will require sampling the available answers over time, and evaluating correctness as a distance from the optimal solution, rather than a binary judgement of correctness.

Analyze the Results, Developing a Theoretic Explanation and Iterating. These three steps are combined because they are largely beyond the scope of the benchmark itself. The researcher is responsible for these, and the benchmark must support them in the next steps. To

that end, the benchmark should clearly report the *efficiency, effectiveness* and *cost* (performance metrics) of each workload on each system under test configuration. It should also report the details of the datasets, implementations and SUT for posterity. Results should be viewable from the perspective of a SUT, (exploring each aspect of performance) and aspect (comparing across SUTs)

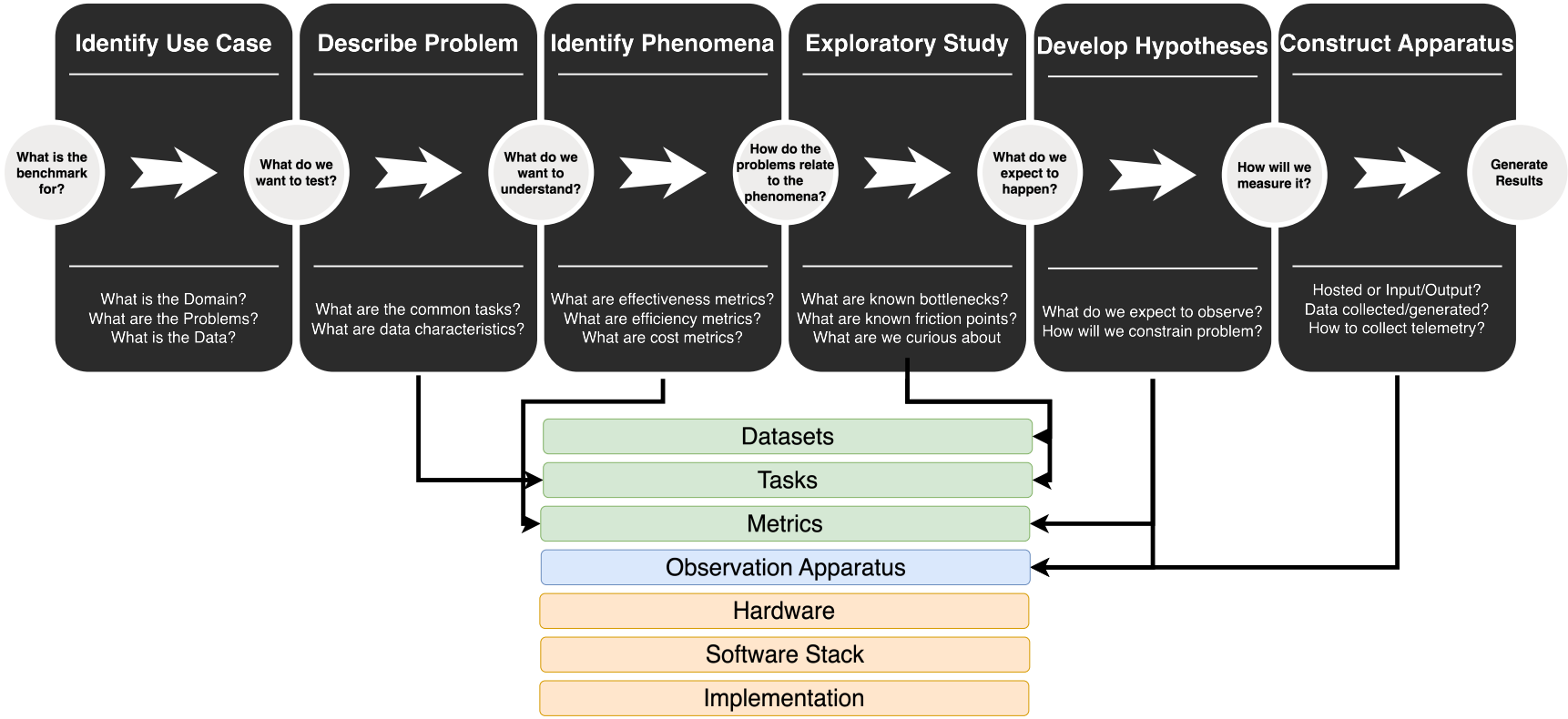


Figure 4.4: GraphAI Benchmark Design Framework

Chapter 5

The Population Segmentation Problem

This chapter demonstrates how the GraphAI benchmark design process described in chapter 4 can be applied to an I&S domain use case to develop a problem for the GraphAI benchmark framework.

The *Identify Use Case* step is not included as a detailed description here because the work in chapter 1 to identify the I&S problems (table 1.2) identified it as a suitable use case. To identify other use cases the same methodology can be applied to other domains.

5.1 Identify and Characterize the Population Segmentation Problem

In this section, we describe the population segmentation problem as it is defined in the I&S domain to identify common tasks and the characteristics of datasets facing real-world system deployment.

5.1.1 Definition

A Population Segmentation Analysis aims to divide a population into groups by geography, demography, attitude toward an issue, or other thematic line. These groups or 'segments' are a subset of the population with common attributes to which an organization can tailor its communication [36]. Beyond its I&S applications to intelligence preparation of the battlespace (IPB), information operations (IO) and civil-military cooperation (CIMIC), population segmentation is

used heavily in marketing and in identifying how to arrange graphs in memory to reduce memory access overheads [143].

Traditionally segmentation is done using data collected by surveys or observation. More recently, similar results have been achieved by mining large volumes of public social media and communications data. For example, prior work has shown how email patterns can predict social subgroups [37] and phone call records reflect geographic and cultural clusters in the real world [38].

These communication networks can be represented as a graph. Each individual is a vertex, and each interaction (phone call, email) is an edge. Weights may reflect the volume or durations of interactions, and attributes for vertices and edges may be used to incorporate additional information.

5.1.2 Tasks

GraphAI tasks that can be used for population segmentation include *Graph Partitioning*, *Graph Clustering*, and more broadly *Community Detection*.

Graph Partitioning takes a *top-down* approach and Graph Clustering a *bottom-up* approach to the task of optimally dividing a graph into K subgraphs such that each vertex belongs to exactly one partition, all partitions are disjoint, and some objective function is optimized. Both graph partitioning and graph clustering have been proven NP-complete. Community Detection is a more general form of the problem that involves the discovery of latent *Communities*, or "subgraphs whose members are more tightly connected than the broader graph" [144]. It is closely related to partitioning and clustering, with the key distinction that communities are a

Problem	Output Format
[1] Who is in the same group as `(x)`?	List of Vertex IDs
[2] Is `(x)` likely to be know `(y)`?	Tuple of (Y/N, Confidence)
[3] Count the vertices in the largest and smallest disjoint segments.	Tuple of (small_size,big_size,[small],[big])
[4] `(x)`, `(y)` and `(z)` are in communities `(a)`, `(b)` and `(c)`. Who else is in `(c)`?	List of Vertex IDs
[5] Who shares a segment with both `(a)` with property `(1)` and `(b)` with property `(2)`?	List of Vertex IDs
[6] Which segment has the most vertices with property `(3)`?	Tuple of (Group_name, [Vertex IDs])
[7] What are the segments and who is in each?	List of Lists of Vertex IDs
[8] There are `(n)` segments, which segment does each vertex belong to?	List of Lists of Vertex IDs

Table 5.1: Example Problems for Population Segmentation

more general structure that do not necessarily enforce the requirements for independence that clusters and partitions do.

Based on the problem survey (section 1.1), available datasets and expected tasks, we identify the initial collection of problems in table 5.1, presented in a general form.

5.1.3 Datasets

Population segmentation focuses on human social networks, which are typically scale-free (see: section 2.2). The community definition reinforces that graph topologies are expected to consist of smaller densely connected subgraphs within the containing graph. At the extremely well-defined end of 'communities,' a graph will consist of cliques (completely connected subgraphs), connected to other cliques by single-edge 'bridges'. At the extremely poorly defined end, a fully connected graph or multigraph has no topological community structure and attributes must be included to provie 'clues' about community structure. The 'fuzzy' community is effectively a uniform random distribution, where the 'closeness' of an individual community is no

greater than that of the graph as a whole.

5.1.4 Outputs

By completing the problem definition step we identify the problems, common tasks and datasets that we use to inform the next steps.

Problems: Table 5.1 formally defines several problems that may involve a single kernel-level task to complete or might require combining several kernels to make a more complicated solution. They are broad enough to encourage creativity, but specific enough that we can evaluate them programmatically.

Tasks: The three most common approaches to population segmentation are Graph Partitioning, Graph Clustering and Community Detection.

Datasets: Typical datasets are Scale-Free, and vary between having well-defined communities (cliques) we expect community detection approaches to perform well on and then fuzzy and topologically bland communities (e.g. fully connected graphs) that will perform poorly. Skewed distributions of community sizes degrade community detection performance.

5.2 Identify and Characterize Phenomena of Interest

We identify the phenomena of interest from the results of the problem survey (table 1.2) that indicate a growing acceptance of trading off a perfectly effective approach that takes a long time to execute with a more efficient approach that might only provide partially correct, or low-resolution solutions. This section describes how we characterize the phenomena of interest as the trade-off between effectiveness, efficiency and cost.

5.2.1 Efficiency

Measurements of efficiency do not vary much from problem to problem in GraphAI . We use the ubiquitous *Execution Time* but add support for *Extract, Transform, Load Time*. Many approaches to population segmentation use iterative algorithms, and so are well suited to providing interim solutions, making the measures of *batch latency* (time from data being sent until a result it generated) and batch rate (average time to produce an interim result).

5.2.2 Effectiveness

There are two ways to evaluate the effectiveness of population segmentation, using ground truth labels or using structural characteristics. Ground truth label-based evaluation is strongly preferred, and Normalized Mutual Information (NMI) [145] is the most commonly used metric to evaluate predicted segments against ground truth. Scalability limitations in NMI are driving alternate methods like Network-F1 (NF1) [146–148] but have not gained widespread traction. Without ground truth labels, the 'goodness' of a segment is calculated using a metric like *modularity* [149], *conductance* [150], or the *Constant Potts Model* [151]. These methods are limited in their ability to provide a reliable evaluation, as they effectively model the probability that the community is more connected than a random graph. Further, optimizing the modularity metric is NP-Complete [8] and heuristic approaches degrade for small communities and skewed community distributions [144], which are typical of scale-free graphs [7, 152].

5.2.3 Cost

Cost is ideally measured as a function of power consumption or as an estimated 'cost per minute' based on the specifications of the SUT. The GraphAI benchmark framework initially focuses on an Input/Output benchmark paradigm, which means the internal states of the system (power consumption, etc) are unknown. Future work is deploying a hosted instance of the benchmark to the ARLIS *Quantum Inspired Classical Computing* (QUICC) testbed. The testbed will provide measurements of power consumption by the controlling docker containers using *Kepler*[1]. Further, we aim to develop a daemon to deploy to a SUT to collect power consumption statistics using the *PowerAPI Project*[2], though this would be an optional component of the benchmark and limit it to running on compatible physical Linux-based systems.

As a result, cost is excluded from the Population Segmentation benchmark problem.

5.2.4 Outputs

By completing the phenomena identification step we determine that we are interested in observing the interaction between the effectiveness and efficiency of approaches, and in particular whether it is possible to trade lower effectiveness for better efficiency. Effectiveness metrics of *NMI* (preferred) or *modularity* (optional) will be used as proxies for effectiveness and efficiency metrics of *ETL Time, Execution Time* and *Batch Latency* will be used as proxies for efficiency. No proxy will be collected for cost at this time.

The phenomena of interest for the initial runs of the benchmark is the relationship between the NMI and batch latency, which should tell us if we can trade effectiveness for efficiency.

[1] `https://sustainable-computing.io/design/architecture/`
[2] `https://powerapi.org/`

5.3 Conduct Exploratory Experimentation

For our exploratory analysis, we conduct a survey of prior work in GraphAI benchmarking, workload analysis and acceleration, which is summarized in table 5.2, grouped broadly by approach. Dynamic approaches are explicitly called out because they tend to operate on temporal graphs, and so have a different data structure to the other approaches. For each approach, we identify the reported worst-case complexity for the algorithm as the efficiency bottleneck, and observations about factors that degrade the quality of communities as effectiveness bottlenecks. Below, we characterize our observations of the stressors to community detection performance: datasets, algorithmic weaknesses, flaws in modularity optimization and the challenges of dynamic graph processing. We finish by identifying two techniques shown to improve performance using external information and approximate solutions.

Family	Algorithm	Efficiency Bottleneck	Effectiveness Botleneck
Graph Partitioning	Kernighan-Lin [153]	$\mathcal{O}(n^3)$	Dense; Small communities; Unknown K
	Extremal Mod. Optimization [154]	$\mathcal{O}(n^2 \log^2 n)$	Fuzzy
	Spectral Partitioning$_{base}$ [155]	$\mathcal{O}((m+n)n)$	Dense
	Spectral Partitioning$_{cl}$ [155]	$\mathcal{O}(n^3)$	Dense
	FEC [156]	$\mathcal{O}(cl(n+m))$	Fuzzy; Many communities; No Attributes
	Potts-Fuzzy [157]	$\mathcal{O}(m+q)$	$L_{in} \gg L_{out}$; Unknown C
Graph Clustering	Spectral Clustering [158]	$\mathcal{O}(n^3)$ [159]	Unknown C
	Girvan-Newman [7]	$\mathcal{O}(n^2 m)$	Fuzzy; Dense
	Hierarchical Agg. [160]	$\mathcal{O}(md \log n)$	Dense; Large C, Small C; Fuzzy
	Hierarchical Div. [149]	$\mathcal{O}((m+n)mn^2)$	Dense; Fuzzy
	Spectral [159]	$\mathcal{O}(nc^2 e)$	Skewed Community Distro; Unknown C
	Louvain [161]	$\mathcal{O}(n^2)$	Many Clique-Bridge Vertices
	Leiden [162]	$\mathcal{O}(n^2)$	Fuzzy
Graph Prediction	DNR [163]	$\mathcal{O}((nh+h^2)+(incd))$	Fuzzy; Unknown C
	Com-E [164]	$\mathcal{O}(n+m)$	Dense; Fuzzy
	M-NMF [165]	$\mathcal{O}(n(m^2 n))$	Dense; Fuzzy; Many C; Unknown C
Dynamic Graphs	Instant Optimal [148, 166]	$\mathcal{O}(T((n^2)+(c^2)))$	Fuzzy; Many C
	Temporal tradeoff [148, 167]	$\mathcal{O}(T(md \log n))$	Many changes per T
	Cross-Time [148, 168]	$\mathcal{O}(3^{\frac{d}{3}} 2^2 mnt)$	Dense

Table 5.2: Summary of Computational Approaches to Population Segmentation

Where: n is number of vertices, m is number of edges, c is number of communities/clusters/partitions, k is degree, t is number of time slices, d is degeneracy of graph, h is number of hidden layers, and i is number of iterations.

5.3.1 Datasets Observations

Graph Size. From the survey in table 5.2 the worst-case complexity is at least quadratic in the number of vertices, particularly for the dominant algorithms like Louvain [161] and Leiden [162]. We can conclude that the number of vertices in the graph will be a driver of efficiency degradation in almost all cases, with many of the approaches vulnerable to dense graph structures where $|E| \gg |V|$.

Graph Shape. Further, we identify that the effectiveness of community detection approaches degrades, unsurprisingly, when the communities are 'fuzzy' rather than 'cliquey', and when the mean degree of the graph is high.

5.3.2 Implementation Observations

Algorithmic Weaknesses exist in Community Detection. Many approaches require the number of communities/clusters/partitions to be known in advance or must run an algorithm like K-Means multiple times to discover the optimal number of communities ([153, 157–159, 163, 165]), significantly increasing the computational cost when the number of communities is large. Knowing the number of communities a priori is not reflective of the discovery process expected in the real world [29]. Recursive Bisection partitioning approaches fail when there are three equally sized communities at a given level of recursion, at least one of them will be mistakenly grouped with another, and will usually be split across both. Greedy Agglomerative clustering approaches fail when two cliques are connected by nodes of low degree compared to the clique, pulling the 'bridge' nodes into the cluster that they bridge to rather than their own. Most approaches are not capable of assigning a vertex to multiple communities because of the underlying assump-

tions about clustering and partitioning, so overlapping communities rely on graph attributes for effective community detection on heterogeneous graph structures [169]. The most popular implementation (Louvain [161]) has been shown to produce disconnected communities when the shape of the input graph contains hierarchical community structures that contain cliques joined by bridge nodes [162].

Modularity Optimization is not a reliable strategy. Modularity has several problems, despite being the dominant metric. Optimizing modularity is NP-Complete [8], so algorithms using modularity as the metric must take heuristic approaches to improve it. It is not scale-independent, that is, a scaling factor where a community with a density of internal edges of the order of the $\sqrt{2|E|}$ times the edges in the graph will not be resolved by modularity optimization. These 'loosely connected' communities are instead mistakenly combined, with the problem most prevalent when the distribution of community sizes have long tails [144].

Dynamic Processing relies on Short Batch Latency. The inputs to dynamic graph processing algorithms are either a collection of related graphs describing the same problem at T time slices or may just have a stream of updates to reflect when a vertex is added, deleted or updates its position in Cartesian space. As a result, the task is typically to identify when a community changes, or which communities persist for long periods of time (the co-traveler problem). Many of the dynamic processing approaches batch the updates and apply a static community detection approach at each iteration then calculate the similarity in communities between each time-step, with more efficient implementations keeping the prior state in memory to reduce computation [148]. These batching approaches fail in real-time settings where the batch latency is greater than the length of each timestep T.

Community Detection is suitable for Approximate and Incremental approaches. Clus-

tering, Partitioning, and Community Detection are typically implemented as iterative algorithms that run until convergence but can return some information at each iteration step. Running an approximate algorithm may provide a less accurate result in a shorter timeframe, and iterative approaches that improve as time proceeds can take longer to converge than approaches. Datasets with hierarchical communities can have correct solutions at different resolution levels.

Weights and Attributes are critical for segmenting fuzzy and overlapping communities. Graph partitioning, clustering, and community detection are traditionally topology-only algorithms. However, access to some states, like the sentiment of an edge [156], or the weights associated with edges [170] yield superior segments. The same is true of attributes, and prior knowledge about group membership of some nodes greatly improves the identification of overlapping and fuzzy communities [171].

5.3.3 Outputs

Completing the exploratory analysis by proxy of a literature review reveals the bottlenecks that impact population segmentation approaches that will guide hypothesis development in the next step.

Efficiency Bottlenecks. I We expect that increasing the number of vertices and the number of edges will both degrade the efficiency of the approaches, given the worst-case complexity estimates in table 5.2

Effectiveness Bottlenecks. The effectiveness of community detection approaches is vulnerable to fuzzy and overlapping communities, denser graphs and when 'bridges' exist between clique-y structures.

Friction Points. Further friction points we identify is the requirement for apriori knowledge of the number of communities by many of the approaches, the lack of support for overlapping communities, and the lack of work in dynamic CD benchmarks.

Curiosities and opportunities. The salient opportunity is to explore the efficiency-effectiveness tradeoff by using an iterative approach to community detection that outputs interim results as they are generated and offers the opportunity to experiment with trading off efficiency and effectiveness. There is also an interesting constraint-based problem to find the optimal Timestep T length for dynamic CD to determine how long it should wait when receiving a stream before commencing a batch update.

5.4 Develop Hypotheses

To construct the hypotheses, we review the context of the use case, the problem and the exploratory analysis to identify the likely interactions that will yield the phenomena we wish to investigate. For the Population Segmentation problem of the GraphAI benchmark, we are particularly interested in the phenomena surrounding the trade-off between effectiveness and efficiency. We combine the specific observations from the previous sections in this chapter with the constraint levers identified in table 4.2 to develop the hypotheses, presenting them grouped by efficiency and effectiveness and constraint-specific tests.

5.4.1 Efficiency Hypotheses

H0: When the number of vertices in a graph increases, the increase in execution time will be no worse than quadratic.

5.4.2 Effectiveness Hypotheses

H1: When the mean degree of a graph increases the NMI will decrease.

H2: When the mean ratio of internal to external community edges (μ) approaches 0.5 (becomes 'fuzzy'), effectiveness will decrease.

H3: When the number of communities c in the ground-truth increases, the NMI will decrease.

H4: Community size distributions following a power-law distribution will have a lower NMI than those following a uniform distribution.

H5: Implementations that use vertex and edge attribute information will have higher NMI than implementations that do not.

H6: Constraining some vertices to be members of specific groups will improve the NMI of attribute-aware approaches

5.4.3 Constraint Hypotheses

H7: As time increases, effectiveness will increase.

H8: There is an optimal time slice length T for dynamic community detection to minimize batch latency.

5.4.4 Outputs

The hypotheses themselves are the outputs of this step and will guide the analysis in section 5.6 and 5.7

5.5 Construct the Investigation Apparatus

The apparatus is the physical instantiation of the problem benchmark. Figure 4.4 shows how the apparatus is derived from each step of the process. Three of the components (Datasets, Tasks and Metrics) are consistent with the approach of traditional benchmarking approaches (see figure 4.1) and we add the explicit definition of the observation apparatus. These four components are the concrete implementation of the benchmark. Combined, they have two key roles. First, they must interact with the SUT, problems and datasets go in, and solutions for evaluation come out. Second, they must monitor the phenomena emerging from the SUT throughout the process. Figure 5.1 is the envisaged workflow for the Population Segmentation problem benchmark, showing how the physical components of the observation apparatus are implemented and how they interact between the benchmark and the SUT. It is representative of the expected workflow for most Problems in the GraphAI Benchmark Framework.

In the remainder of this section, we summarize the concrete components of the apparatus, showing how they were created and identifying expansion opportunities.

5.5.1 Observation Apparatus

The observation apparatus is a component we add to our benchmark design to explicitly represent the decisions about how the benchmark will interact with the SUT. We choose to implement an Input/Output benchmark, augmented with constraint-based hypotheses testing. While the Input/Output approach prevents us from monitoring the internal state of the system, it is maximally accommodating of researchers altering the hardware, software stack and implementation. System variance reflects the real-world implications of the benchmark, if someone can afford to buy and run expensive hardware to achieve better results, they are free to do so.

More specifically we choose the Input/Output model to accommodate the huge variety in GraphAI acceleration approaches. Hardware accelerator designs [14,68,73,80,125,127,129,132,134,135,172,173], GNN accelerator designs [20,115,137,139,173–181] are proliferating rapidly. Implementations exist for canonical sequential [161], distributed [182], shared-memory [183] on CPU and GPU [73], as well as linear-algebraic approaches for CPU [184] and GPU [185] for community detection approaches. Configuring instrumentation for all of these systems is infeasible, and a hosted benchmark would exclude too many of the approaches, leading us to an enhanced Input/Output model for the benchmark.

5.5.2 Datasets

Datasets in the GraphAI population segmentation problem are expected to be large, scale-free and heterogeneous, attributed graphs. The hypotheses identify the requirement to scale the number of vertices, the density of the graphs, the distribution of degrees, the distribution of communities and optionally include weights and attributes. The review of Community Detection

problems also shows that community detection can be applied to dynamic graphs.

To stress across these parameters, we first look for datasets in the *Tiny, Small, Medium* and *Large* range. We define Tiny datasets as those that fit in the cache of a commercial system, < 60Mb[3]). Small can be processed in memory on a personal computer, no larger than ~ 16Gb, the medium should fit in the memory of a commercial system, no larger than ~ 1Tb and large should be bigger than what will fit in the memory of a single commercial system (at least ~ 5Tb).

Building on the review of dataset coverage in table 5.3, we should aim for full coverage of scale-free, uniform-random and mesh shapes. To enable evaluation, all datasets require ground-truth communities/clusters/partitions for NMI calculation. To test the extra stress on memory, and provide the opportunity for approaches to leverage the information in their solutions, some datasets should be weighted and attributed, and partial assignments of vertices to communities should be available. There should be dynamic graphs suitable for batch updating and continuous streaming so that dynamic processing algorithms can be evaluated alongside static solutions. Finally, to stress effectiveness, graphs should be available with varying community distributions, mean degree, 'fuzziness' and overlapping and nested communities.

Our datasets are summarized in table 5.3. We identify seven suitable datasets from the SNAP repository ranging from tiny to medium in size are scale-free in shape and cover the distribution of community sizes from small to large. Noting the lack of mesh and random graphs, we generate our own.

[3]Based on Intel Xeon Gold `https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable/gold/products.html`

5.5.2.1 Mesh Graph Generation

There were no publicly available mesh graphs with ground truth community labels. We develop a tool to generate directed, weighted, attributed graphs with hierarchical and overlapping communities from OpenStreetMap (OSM) road network data[4]. Our tool queries the OSM API for a given city, county or state, extracts the road segments (ways) and creates a vertex where the ends of two segments meet or where two segments intersect. We construct the graph using Python's NetworkX [186] and assign each vertex the attributes of its latitude, longitude and the name of the street it comes from. We group the vertices by subordinate communities. For example, a state has community groups at the county and city levels. Where a road crosses a border, it is assigned to a special group that is a member of both communities. We use the tool to create six tiny and small datasets with very low density, wide diameters, and a broad distribution of community counts across multiple resolutions.

5.5.2.2 Random Graph Generation

While random graphs are not an effective substitute for real-world graphs from the target domain, they are a useful supplemental tool to cover gaps in real-world data and to develop datasets to stress specific system components. We lack coverage of the spectrum of graph shapes between mesh and scale-free. Imitative generators are not yet at the maturity to generate the medium and large datasets we require, nor are suitable statistical summaries of graphs available to seed imitative generators. Specific community-detection generators like LFR [187], AcMARK [188], GenCAT [189] can generate graphs with ground-truth communities following

[4]Tool available at `https://github.com/osullik/umd_thesis/blob/main/md_roads/osmg.py`

mathematical distributions.

Source	Dataset	Size	Vertices	Edges	Communities	Med Deg	Topo	D	A	W	T	Domain/Model
Real-World	Karate Club [37, 120]	500b	34	78	2	50	SF					
(SNAP)	Cisco Networks$_{g_{21}}$ [190]	2Mb	317	3025	23			✓			✓	Computer Network
	Cisco Networks$_{g_{22}}$ [190]	34.9Mb	33,241	702,361	10			✓			✓	Computer Network
	Email Eu Core [191]	200Kb	1005	25571	42		SF	✓				Emails
	Orkut [192]	1.77Gb	3,072,441	117,185,083	6,288,363		SF					Social Media
	Friendster [192]	32.36Gb	65,608,366	1,806,067,135	957,154		SF					Social Media
	Wiki-topcats [191]	46.6Mb	1,791,489	28,511,807	17,364		SF					Wiki Hyperlinks
Real-World	OSM-M	240Kb	2063	2572	4; 2	2	mesh	✓	✓	✓		Roads
(Mine)	OSM-DC	15.2Mb	147,599	163,156	27; 10	2	mesh	✓	✓	✓		Roads
	OSM-DE	63.5Mb	637,463	680,858	63; 17	2	mesh	✓	✓	✓		Roads
	OSM-RI	65.2Mb	671,727	711,157	63; 4	2	mesh	✓	✓	✓		Roads
	OSM-MD	441.6Mb	4,513,407	4,745,260	402; 108	2	mesh	✓	✓	✓		Roads
	OSM-WA	665.3Mb	6,690,579	6,996,063	430; 108	2	mesh	✓	✓	✓		Roads
Synthetic												

Table 5.3: Summary of Datasets for the Population Segmentation problem

D is Directed, **A** is Attributed, **W** is weighted, and **T** is Temporal. **Size** is the sum of the uncompressed adjacency list and attribute file. All datasets have a ground-truth community. Where different community resolutions exist, both are listed (e.g. city; county) for the OSM graphs. Cross-community nodes are permitted.

5.5.3 Tasks

We draw on the problems identified in table 5.1 for our specific tasks. To enhance the benchmark, we intend for the tasks the be publicly, and generically specified, with specific values provided only at experiment time, in an attempt to reduce researchers 'gaming' the system. Figure 5.1 shows the back-and-forth process between the SUT and the benchmark as steps $1-4$.

5.5.4 Metrics

Using the Input/Output paradigm, telemetry can only be gathered by observing what data is passed into a system, what data comes out of a system, and how much time elapses between. Figure 5.1 shows what the Input/Output operations and internal processes look like for the population segmentation problem between a System Under Test and Benchmark Server (BS). ETL and Processing are shown linearly but could be interleaved in the case a SUT chooses to take an incremental approach. The shaded region denotes the polling period. During the polling period, the BS will periodically poll the SUT for its current answer, and log the time and content of the response, identifying where partial or approximate answers are available ahead of the algorithm officially 'completing'. At each step of the process, the time is logged by the BS for generating the efficiency metrics *Transfer Time, ETL Time, Execution Time, Batch Latency* and *Batch Rate*. The Effectiveness metric for all problems requiring the comparison of communities in ground truth and predicted nodes is *NMI*.

5.5.5 Limitations

Several limitations exist in the current design of the population segmentation problem benchmark. First, transferring the data is highly sensitive to network bandwidth and latency, disadvantaging remote researchers. Using static datasets can lead to gaming the system, which is why we implement the randomized problem parameters, hash checks and network data summaries to verify that the benchmark process is being followed. It is still possible to game it, but it requires a more deliberate effort.

5.5.6 Outputs

The apparatus construction step yields the dataset observation apparatus and GraphAI problem instance that is used for testing and will produce the experiment results.

5.6 Analyze Results, Develop Theoretic Explanation and Iterate

To illustrate the utility of the benchmark, we conduct a small-scale demonstration, comparing a single sample problem on eight real-world datasets. First, we describe the specifics of the experiment setup and second, we describe the results.

5.6.1 Demonstration Setup

We implement the GraphAI population segmentation benchmark in Python 3.11.5 and make the environment[5] details available. The implementation follows the design from figure 5.1, with the exception that the benchmark server is located on the system under test and that the polling is

[5] https://github.com/osullik/umd_thesis/tree/main

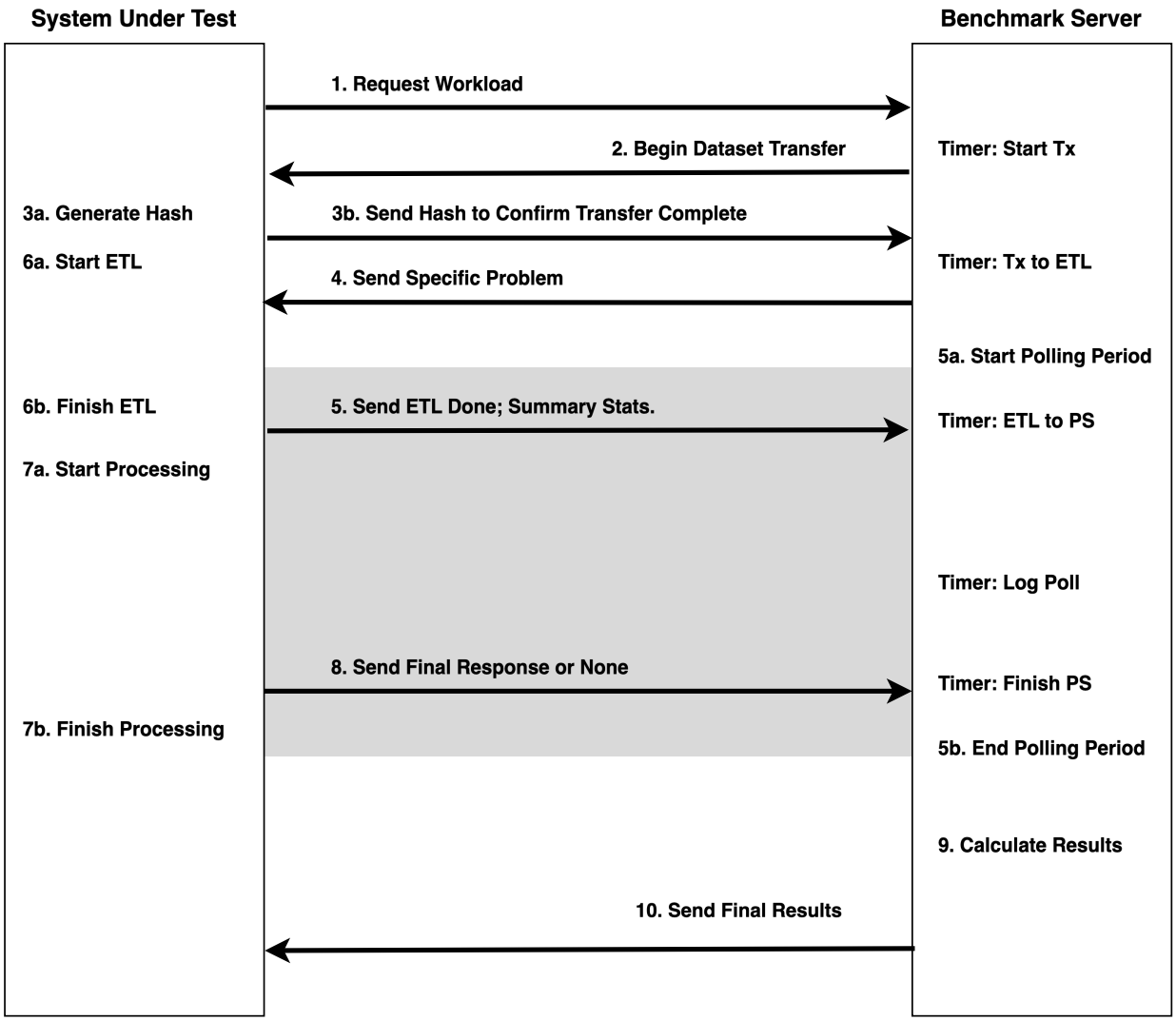


Figure 5.1: Community Detection Problem Apparatus

done passively by checking the timestamps of an interim-results folder after the experiment. To run the benchmark, the researcher just specifies the problem and dataset.

We provide experiment specifications for the 28 experiments listed in table 5.4 as `json` files[6] that the `benchmark server` uses to create the experiments. To maximize usability, the process is completely automated from requesting a benchmark problem from the server until the server provides the data and figures in the return report. The experiments are run on a single

[6]available at: `https://github.com/osullik/umd_thesis/tree/main/gaib/experiment_definitions`

#	Dataset	Problem	Return Format	Approach
1	OSM-M (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
2	OSM-M (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
3	OSM-M (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
4	OSM-M (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
5	OSM-DC (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
6	OSM-DC (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
7	OSM-DC (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
8	OSM-DC (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
9	OSM-DE (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
10	OSM-DE (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
11	OSM-DE (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
12	OSM-DE (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
13	OSM-RI (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
14	OSM-RI (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
15	OSM-RI (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
16	OSM-RI (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
17	OSM-MD (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
18	OSM-MD (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
19	OSM-MD (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
20	OSM-MD (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
21	OSM-WA (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
22	OSM-WA (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
23	OSM-WA (City)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
24	OSM-WA (County)	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
25	Email-EU-Core	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
26	Email-EU-Core	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental
27	Wiki-Topcats	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Static
28	Wiki-Topcats	*Who are the members of each segment?*	List of Vertex ID Lists (by segment)	Incremental

Table 5.4: Summary of Experiments for CD Demonstration

configuration of hardware and supporting software[7], with only the implementation varying in the approach to the problem. We limit our choice of implementations to the NetworkX library to standardize the representation and ETL processes between implementations. The static approach uses the NetworkX *Louvain Communities*[8] implementation and the incremental use the NetworkX *Louvain Partitions*[9] library, which iteratively yields communities at decreasing granularity.

[7] 2023 Apple Macbook Pro; 12 Core M2 Max Processor, 32GB Hynix LPDDR5 RAM; MacOS 14.1-Darwin-23.4.0

[8] `https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.louvain.louvain_communities.html`

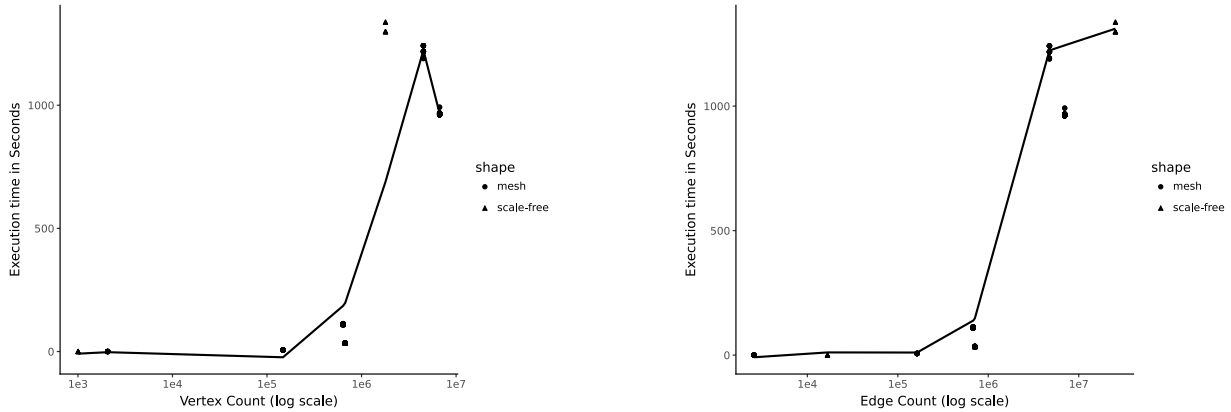
[9] `https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.community.louvain.louvain_partitions.html`

5.6.2 Demonstration Results

We tie the evaluation closely to the Hypotheses from section 5.4 and so group the results by efficiency, effectiveness and Hypotheses are used as a guide for investigation rather than a formal testing mechanism, so we abstain from statistical analysis of results at this time, but intend to automate evaluation against the hypotheses as part of future work.

5.6.2.1 Efficency Results

The total experiment time across all experiments is dominated by the execution time (figure 5.3) which scales quadratically in the number of vertices (figure 5.2). The batch latencies are equivalent for both the static and incremental approaches.



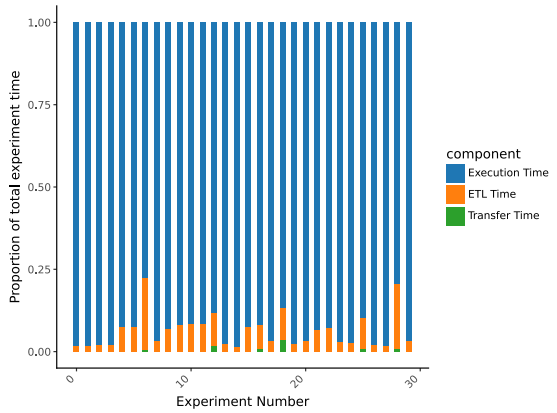
(a) Execution time is ∼quadratic in vertex count

(b) Execution time is ∼quadratic in edge count

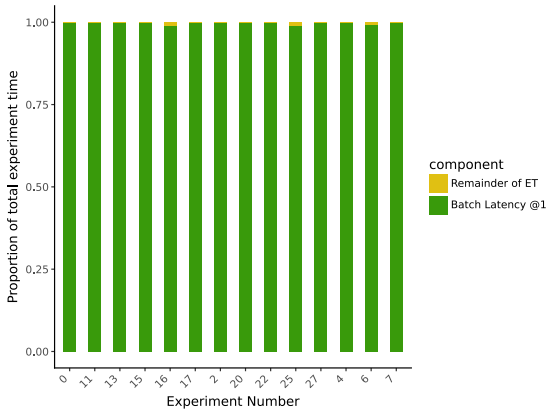
Figure 5.2: Efficiency results for Community Detection show quadratic time complexity.

5.6.2.2 Effectiveness Results

Examining the number of predicted versus ground truth communities shows that NMI increases as the number of predicted communities increases, but that there is a concurrently in-



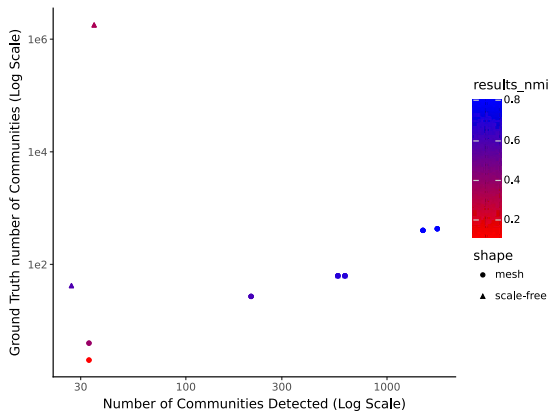
(a) Execution time dominates total experiment time



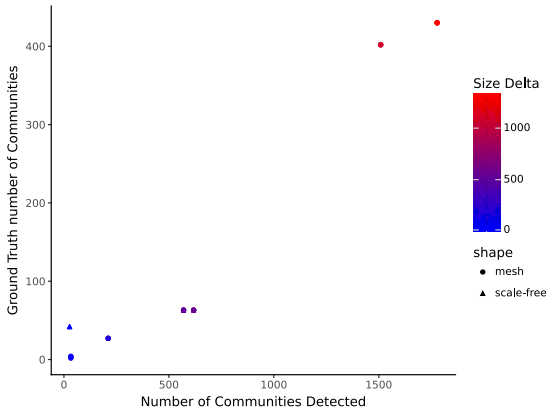
(b) The incremental approach does not yield incrementally. The first batch completes at 99% of Experiment Time

Figure 5.3: Execution time dominates the Louvain approaches.

creasing delta between the number of predicted and ground truth communities (figure 5.4). Both of these covary with the size of the graph. Examining the effectiveness of the iterative algorithm over each increment in Figure 5.5 shows that while the NMI converges to a higher value and the number of communities decreases as iterations continue they do not 'improve' monotonically over time.

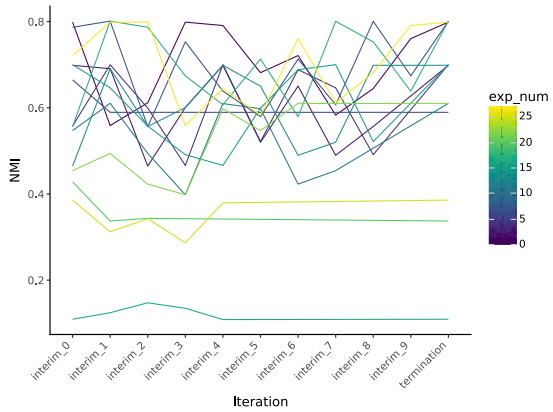


(a) NMI appears to improve as the number of communities increases.

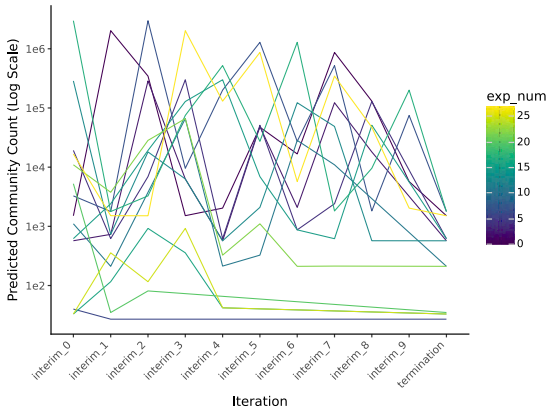


(b) Community Size mispredictin grows as graph size does. Values closer to zero are better. (outlier excluded)

Figure 5.4: NMI appears disconnected from predicting the correct number of communities.



(a) NMI Behaves non-monotonically in iterative approaches but converges to a higher value.



(b) Estimates of the number of predicted communities vary non-monotonically between increments and typically finish with fewer communities. Graphs with fewer communities converge with fewer iterations.

Figure 5.5: The incremental approach is erratic, varying the number and quality of communities at each iteration.

5.6.2.3 Constraint Results

In addition to the observations about the relationship between efficiency and effectiveness of the incremental approach about figure 5.5, there is no statistically significant difference between the distribution of NMI at each iteration (figure 5.6).

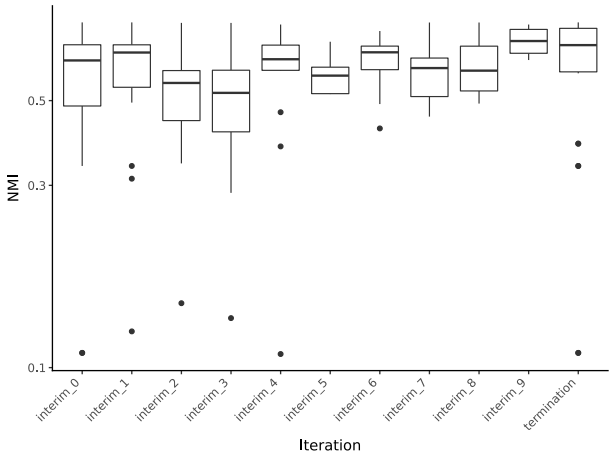


Figure 5.6: NMI varies at each iteration with no interim step being significantly different from the result at termination

5.7 Discussion

Based on the results described in section 5.6 and the context of the hypotheses stated in section 5.4 we make the following observations.

The NetworkX implementation of the Louvain algorithm has quadratic complexity in practice. Execution time increases quadratically in both the number of vertices and the number of edges. Our datasets are weighted towards mesh graphs where the number of $|E| \rightarrow |V|$. While this is not representative of real-world community structures, it does show that the assertion that the Louvain algorithm and its derivatives are 'near linear in practice [161, 162] is misleading.

Execution time dominates, but ETL time is non-trivial for small graphs. Benchmarks focus on execution time because it is the bottleneck of a system. For small graphs, ETL can take up to 20% of total experiment time. As we implement support for dynamic graphs which involve many small ETL activities over time, we expect to see ETL taking larger proportions of the total experiment runtime.

The Louvain Partitions implementation does not yield incremental solutions. Reviewing the batch latency (figure 5.3b) shows that the NetworkX implementation of Louvain Partitions does not yield an answer after each partition. Instead, it completes the entire algorithm and then returns a generator that contains all of the partitions. To effectively measure the 'benefit' of an incremental solution through batch latency, an incremental algorithm must return results in real-time, or at least in batches smaller than 99% of the total runtime.

NMI is an unreliable metric for evaluating population segmentation. NMI uniformly increases as the number of predicted communities increases, skewing the 'goodness' evaluation. The issue is apparent when comparing the number of predicted communities (figure 5.5b) to the

NMI (figure 5.5a) for the same experiment. High NMI scores correlate with large deltas between the predicted number of communities and the actual number of communities, indicating that they cannot be the 'good' result NMI suggests.

The Louvain Partition implementation does not improve monotonically over time. One of the assumptions about the benefit of an incremental or approximate approach for trading off between effectiveness and efficiency is that it should improve monotonically. That is, the longer it runs, the better the solution gets. Figures 5.5, 5.4 and 5.6 show erratic variation in the size of communities and the NMI at each step, suggesting that we cannot make predictable tradeoffs of efficiency and effectiveness. Adding more datasets and different incremental implementations will assist in understanding this behavior, which we believe is the biggest impediment to trading off between efficiency and effectiveness.

Summary. Based on our small experiment, we draw preliminary conclusions that these approaches are not suitable for trading off effectiveness for efficiency. We identify the need to improve the balance in our datasets between mesh, scale-free and random; find incremental implementations that periodically output solutions; identify a better metric than NMI for evaluating effectiveness and find methods that monotonically improve community quality over time. The first three are benchmark design considerations, but the last one is an algorithmic design question—does a community detection algorithm exist that improves monotonically, and if not, how could we construct one?

Chapter 6

Conclusion

6.1 Future Work

The GraphAI benchmark framework is in its infancy and has many avenues for future work. Many are engineering and data collection problems to build out the benchmark system, including rectifying the deficiencies in the dataset coverage, choice of metric and breadth of implementations the Population Segmentation experiments found. Specific engineering challenges of immediate interest are adding a linear algebraic approach to community detection, and testing the benchmark on distributed, shared-memory parallel and GPU-accelerated systems.

A benchmarking Maturity Model. In reviewing many benchmark approaches suggests an implicit maturity model in benchmark development. For a given field, early benchmarks focus on proving functionality. Over time, the focus shifts to improving the effectiveness of the solutions of others, and then finally focus shifts to making solutions more efficient. Poorly constructed maturity models have gained negative attention recently [193, 194] and so the formal development of a maturity model for benchmarking was scoped out of this thesis but is useful for future work to investigate the correlation between benchmark availability and field maturity.

Monotonically Improving Incremental Algorithms. The Population Segmentation benchmark problems reveal that trading off effectiveness and efficiency has an implicit requirement of monotonically improving solutions for incremental and approximate algorithms. Further work must identify existing algorithms with these guarantees, and for problems where none exist, the

real-world drivers identified in this thesis provide a strong incentive to develop them.

Automated hypothesis Evaluation. Our design process involves the development of hypotheses to guide the design of the investigation apparatus and collection of data. Encoding the hypotheses so that they alert when a benchmark submission violates their assumptions is a useful augmentation to the current approach. Violating the 'conventional wisdom' they represent likely indicates an innovative approach at best or a manipulation of the system at worst. Improving the automated evaluation will boost the utility of the benchmark, and move it away from being a tool for 'racing systems' towards a platform for meaningful research.

Non-invasive measurement of Cost. The major limitation of the GraphAI benchmark framework is the inability to accurately measure the cost of a system. Current approaches focus on estimation using a cost model, or selecting a proxy metric like power consumption as the basis of cost estimates. To add the third crucial dimension to the efficiency and effectiveness metrics already collected, we need to develop a method to measure cost from the Input/Output interface. Alternatively, developing a method to characterize a remote system so that the benchmark framework can estimate costs for the system under test may be feasible, but is limited by the use of bespoke hardware accelerators that may not support a general-purpose tool running on their system to collect that information.

6.2 Conclusion

This thesis develops a benchmarking framework for GraphAI . Chapter One identifies GraphAI as a field of growing importance with the emergence of concepts like neuro-symbolic artificial intelligence and retrieval augmented generation. It characterizes a taxonomy spanning

from classical AI problems in searching and planning, through structural analysis, reasoning and the development of Graph Neural Networks. We show how for the Intelligence and Security domain, GraphAI approaches can address many of their problems, and that deploying them to the real world imposes temporal, financial, SWaP and accuracy constraints. Frequently one system will be expected to solve many GraphAI problems, and so determining which system provides the best balance of efficiency, effectiveness and cost across its problem sets is an important question in procurement. The go-to tool for understanding the relative performance of systems is benchmarking.

Chapter Two explains how the proliferation of distinct graph abstraction models, representations, storage formats, primitive operations, and programming frameworks, together with the heavily data-dependent character of GraphAI tasks and the ineffectiveness of modern general-purpose architectural optimizations for graph data structures makes them very difficult to evaluate.

From Chapter Three we identify that a useful benchmark needs to do more than just 'race' systems against each other on a leaderboard and introduce empirical and constraint-based benchmarking as alternate approaches that produce more meaningful results. Reviewing existing benchmarks shows a lack of coverage for dynamic graph processing, a focus on kernel-level tasks at the expense of a holistic 'problem-centric' view and a focus on either efficiency or effectiveness but never both. We show how prior workload analyses and efforts to accelerate GraphAI hardware provide detailed insights into the internal states of GraphAI systems, and conclude that while the instrumentation and profiling required make these approaches infeasible for a large-scale benchmark, the data dependence of GraphAI tasks means we can manipulate the input to stimulate some of the same behaviors.

The findings from these three chapters yield the GraphAI benchmark framework in Chapter Four. The framework is problem-centric. The general framework provides a common interface for problems to be developed and integrated. The framework design and abstract development methodology we develop is applied to the Population Segmentation problem from the Intelligence and Security domain as an exemplar use-case in Chapter Five. By stepping through the process, we develop the Population Segmentation Benchmark for the GraphAI benchmark framework. In addition to the methodological contribution, we describe the development of a tool to convert geospatial data from OpenStreetMaps to directed, weighted, attributed graphs with ground-truth community labels to fill a gap in publicly available mesh graphs for community detection. We also provide a prototype implementation of the observation apparatus and run 28 discrete experiments to demonstrate the functionality of the Population Segmentation Benchmark Problem as a component of the GraphAI benchmark. Analyzing the results of those experiments shows that the ability to trade off the effectiveness of a solution for a more efficient one is dependent on the existence of incremental or approximate algorithms that improve effectiveness monotonically over time, and shows that Normalized Mutual Information is an unreliable metric for evaluating the effectiveness of a community detection approach to solving the population segmentation problem.

Overall, this thesis shows that a clear methodology, grounded in prior work, provides a mechanism to develop a benchmark that goes beyond 'racing' systems against each other towards developing a deeper understanding of a system under test without needing to advance beyond the Input/Output boundary. The formal methodology gives a clear path for other problems to be added to the benchmark and promotes a transparent, reproducible approach to the scientific inquiry that needs to happen to continue advancing GraphAI .

Bibliography

[1] John N Hooker. Needed: An empirical science of algorithms. *Operations research*, 42(2):201–212, 1994.

[2] John N Hooker. Testing heuristics: We have it all wrong. *Journal of heuristics*, 1:33–42, 1995.

[3] William Regli, Ben Johnson, Laurel Miller-Sims, Ali Keshavarzi, and Mohammad Teli. Software-defined hardware:a study of emerging hardware for ai applications. Technical report, 2022.

[4] Stuart Russell and Peter Norvig. Artificial intelligence: a modern approach, 4th us ed. *University of California, Berkeley*, 2021.

[5] Li Zeng, Yan Jiang, Weixin Lu, and Lei Zou. Deep analysis on subgraph isomorphism. *arXiv preprint arXiv:2012.06802*, 2020.

[6] Kent O'Sullivan, Nicole R Schneider, and Hanan Samet. Compass: Cardinal orientation maniulation and pattern aware spatial search. In Li Hao, Gabriele Cavallaro, Dora Blanco Heras, Dalton Lunga, Martin Werner, and Andreas Züfle, editors, *GeoSearch'23: Proceedings of the 2nd ACM SIGSPATIAL International Workshop on Searching and Mining Large Collections of Geospatial Data*, 2023.

[7] Michelle Girvan and Mark EJ Newman. Community structure in social and biological networks. *Proceedings of the national academy of sciences*, 99(12):7821–7826, 2002.

[8] Ulrik Brandes, Daniel Delling, Marco Gaertler, Robert Görke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. Maximizing modularity is hard. *arXiv preprint physics/0608255*, 2006.

[9] Lawrence Page. The pagerank citation ranking: Bringing order to the web. technical report. *Stanford Digital Library Technologies Project, 1998*, 1998.

[10] Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of mathematical sociology*, 25(2):163–177, 2001.

[11] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. `http://snap.stanford.edu/data`, jun 2014.

[12] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and S Yu Philip. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE transactions on neural networks and learning systems*, 33(2):494–514, 2021.

[13] Alexander Maedche and Steffen Staab. Ontology learning. In *Handbook on ontologies*, pages 173–190. Springer, 2004.

[14] Xiaojun Chen, Shengbin Jia, and Yang Xiang. A review: Knowledge reasoning over knowledge graph. *Expert Systems with Applications*, 141:112948, 2020.

[15] Antoine Bosselut, Hannah Rashkin, Maarten Sap, Chaitanya Malaviya, Asli Celikyilmaz, and Yejin Choi. Comet: Commonsense transformers for automatic knowledge graph construction. *arXiv preprint arXiv:1906.05317*, 2019.

[16] Aleksandr Perevalov, Xi Yan, Liubov Kovriguina, Longquan Jiang, Andreas Both, and Ricardo Usbeck. Knowledge graph question answering leaderboard: A community resource to prevent a replication crisis. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 2998–3007, 2022.

[17] Aditya Grover and Jure Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.

[18] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014.

[19] Yukuo Cen, Zhenyu Hou, Yan Wang, Qibin Chen, Yizhen Luo, Zhongming Yu, Hengrui Zhang, Xingcheng Yao, Aohan Zeng, Shiguang Guo, et al. Cogdl: A comprehensive library for graph deep learning. In *Proceedings of the ACM Web Conference 2023*, pages 747–758, 2023.

[20] Trinayan Baruah, Kaustubh Shivdikar, Shi Dong, Yifan Sun, Saiful A Mojumder, Kihoon Jung, José L. Abellán, Yash Ukidave, Ajay Joshi, John Kim, and David Kaeli. Gnnmark: A benchmark suite to characterize graph neural network training on gpus. In *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 13–23, 2021.

[21] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, 32(1):4–24, 2020.

[22] Hao Yuan, Jiliang Tang, Xia Hu, and Shuiwang Ji. Xgnn: Towards model-level explanations of graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 430–438, 2020.

[23] Bastian Pfeifer, Anna Saranti, and Andreas Holzinger. Gnn-subnet: disease subnetwork detection with explainable graph neural networks. *Bioinformatics*, 38(Supplement_2):ii120–ii126, 2022.

[24] Doug Lenat and Gary Marcus. Getting from generative ai to trustworthy ai: What llms might learn from cyc. *arXiv preprint arXiv:2308.04445*, 2023.

[25] Wenhao Yu. Retrieval-augmented generation across heterogeneous knowledge. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Student Research Workshop*, pages 52–58, 2022.

[26] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, and Bin Cui. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473*, 2024.

[27] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, pages 1–20, 2024.

[28] Dan Hendrycks, Nicholas Carlini, John Schulman, and Jacob Steinhardt. Unsolved problems in ml safety. *arXiv preprint arXiv:2109.13916*, 2021.

[29] William Regli, Peter Loats, Laurel Miller-Sims, Ben Johnson, Jacob Bunker, Jared Ott, and Christine Herlihy. Operationalizing emerging hardware for ai applications: A survey of transition opportunities and datasets. Technical report, 2022.

[30] Jennifer J Xu and Hsinchun Chen. Fighting organized crimes: using shortest-path algorithms to identify associations in criminal networks. *Decision Support Systems*, 38(3):473–487, 2004.

[31] Abdullah Sawas, Abdullah Abuolaim, Mahmoud Afifi, and Manos Papagelis. A versatile computational framework for group pattern mining of pedestrian trajectories. *GeoInformatica*, 23:501–531, 2019.

[32] Andreas Tritsarolis, George-Stylianos Theodoropoulos, and Yannis Theodoridis. Online discovery of co-movement patterns in mobility data. *International Journal of Geographical Information Science*, 35(4):819–845, 2021.

[33] Lu-An Tang, Yu Zheng, Jing Yuan, Jiawei Han, Alice Leung, Wen-Chih Peng, and Thomas La Porta. A framework of traveling companion discovery on trajectory data streams. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 5(1):1–34, 2014.

[34] Xiongbin Wang, Chen Liu, and Meiling Zhu. Instant traveling companion discovery based on traffic-monitoring streaming data. In *2016 13th Web Information Systems and Applications Conference (WISA)*, pages 89–94. IEEE, 2016.

[35] Kent O'Sullivan, Nicole R Schneider, Aleeza Rasheed, and Hanan Samet. Gestalt: Geospatially enhanced search with terrain augmented location targeting. In Li Hao, Gabriele Cavallaro, Dora Blanco Heras, Dalton Lunga, Martin Werner, and Andreas Züfle, editors, *GeoSearch'23: Proceedings of the 2nd ACM SIGSPATIAL International Workshop on Searching and Mining Large Collections of Geospatial Data*, 2023.

[36] Department of the Army. *ADP 3-13 Information*. Department of Defense, 2023.

[37] Wayne W Zachary. An information flow model for conflict and fission in small groups. *Journal of anthropological research*, 33(4):452–473, 1977.

[38] Stanislav Sobolevsky, Riccardo Campari, Alexander Belyi, and Carlo Ratti. General optimization technique for high-quality community detection in complex networks. *Physical Review E*, 90(1):012811, 2014.

[39] Shima D Keene et al. Operationalizing counter threat finance strategies. *The Letort Papers*, 2014.

[40] Anastasios Balaskas and Virginia NL Franqueira. Analytical tools for blockchain: Review, taxonomy and open challenges. In *2018 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, pages 1–8. IEEE, 2018.

[41] Daniel Goldsmith, Kim Grauer, and Yonah Shmalo. Analyzing hack subnetworks in the bitcoin transaction graph. *Applied Network Science*, 5(1):1–20, 2020.

[42] Krzysztof Michalak and Jerzy Korczak. Graph mining approach to suspicious transaction detection. In *2011 Federated conference on computer science and information systems (FedCSIS)*, pages 69–75. IEEE, 2011.

[43] Reza Soltani, Uyen Trang Nguyen, Yang Yang, Mohammad Faghani, Alaa Yagoub, and Aijun An. A new algorithm for money laundering detection based on structural similarity. In *2016 IEEE 7th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pages 1–7, 2016.

[44] TRADOC Intelligence Support Activity-Threats Fort Leavenworth KS. *A Military Guide to Terrorism in the Twenty-First Century*. Department of Defense, 2007.

[45] Eric M Hutchins, Michael J Cloppert, Rohan M Amin, et al. Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research*, 1(1):80, 2011.

[46] Lukáš Sadlek, Pavel Čeleda, and Daniel Tovarňák. Identification of attack paths using kill chain and attack graphs. In *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pages 1–6. IEEE, 2022.

[47] Kent O'Sullivan and Benjamin Turnbull. The cyber simulation terrain: Towards an open source cyber effects simulation ontology. In *Proceedings of the 11th Annual Australian Information Warfare Conference*. Security Research Institute, Edith Cowan University, 2015.

[48] Department of the Army. *ADP 2-0 Intelligence*. Department of Defense, 2019.

[49] Pedro Szekely, Craig A Knoblock, Jason Slepicka, Andrew Philpot, Amandeep Singh, Chengye Yin, Dipsy Kapoor, Prem Natarajan, Daniel Marcu, Kevin Knight, et al. Building and using a knowledge graph to combat human trafficking. In *The Semantic Web-ISWC 2015: 14th International Semantic Web Conference, Bethlehem, PA, USA, October 11-15, 2015, Proceedings, Part II 14*, pages 205–221. Springer, 2015.

[50] Yulei Wu, Hong-Ning Dai, and Haina Tang. Graph neural networks for anomaly detection in industrial internet of things. *IEEE Internet of Things Journal*, 9(12):9214–9231, 2021.

[51] Lingfei Ren, Ruimin Hu, Dengshi Li, Yang Liu, Junhang Wu, Yilong Zang, and Wenyi Hu. Dynamic graph neural network-based fraud detectors against collaborative fraudsters. *Knowledge-Based Systems*, 278:110888, 2023.

[52] Mark S Granovetter. The strength of weak ties. *American journal of sociology*, 78(6):1360–1380, 1973.

[53] Daniel Carmody, Martina Mazzarello, Paolo Santi, Trevor Harris, Sune Lehmann, Timur Abbiasov, Robin Dunbar, and Carlo Ratti. The effect of co-location on human communication networks. *Nature Computational Science*, 2(8):494–503, 2022.

[54] Micaela Lucia Bangerter, Giuseppe Fenza, Mariacristina Gallo, Vincenzo Loia, Alessandra Petrone, and Alberto Volpe. Terrorist organization identification using link prediction over heterogeneous gnn. *Human-centric Computing and Information Sciences. 12: 1-13 (www. hcisjournal. com)*, 2022.

[55] a Ahmadi, Hoang H Nguyen, Zijian Zhang, Dmytro Bozhkov, Daniel Kudenko, Maria Jofre, Francesco Calderoni, Noa Cohen, and Yosef Solewicz. Inductive and transductive link prediction for criminal network analysis. *Journal of Computational Science*, page 102063, 2023.

[56] John L. and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6 edition, 2019.

[57] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *science*, 286(5439):509–512, 1999.

[58] Scott Beamer, Krste Asanovic, and David Patterson. Locality exists in graph processing: Workload characterization on an ivy bridge server. In *2015 IEEE International Symposium on Workload Characterization*, pages 56–65. IEEE, 2015.

[59] Paul Erdős, Alfréd Rényi, et al. On the evolution of random graphs. *Publ. math. inst. hung. acad. sci*, 5(1):17–60, 1960.

[60] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. R-mat: A recursive model for graph mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining*, pages 442–446. SIAM, 2004.

[61] Jure Leskovec, Deepayan Chakrabarti, Jon Kleinberg, Christos Faloutsos, and Zoubin Ghahramani. Kronecker graphs: an approach to modeling networks. *Journal of Machine Learning Research*, 11(2), 2010.

[62] Kiarash Zahirnia, Yaochen Hu, Mark Coates, and Oliver Schulte. Neural graph generation from graph statistics. *Advances in Neural Information Processing Systems*, 36, 2024.

[63] Aydin Buluç, Jeremy T Fineman, Matteo Frigo, John R Gilbert, and Charles E Leiserson. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In *Proceedings of the twenty-first annual symposium on Parallelism in algorithms and architectures*, pages 233–244, 2009.

[64] Julian Shun, Laxman Dhulipala, and Guy E Blelloch. Smaller and faster: Parallel processing of compressed graphs with ligra+. In *2015 Data Compression Conference*, pages 403–412. IEEE, 2015.

[65] Robert Ryan McCune, Tim Weninger, and Greg Madey. Thinking like a vertex: A survey of vertex-centric frameworks for large-scale distributed graph processing. *ACM Computing Surveys (CSUR)*, 48(2):1–39, 2015.

[66] Grzegorz Malewicz, Matthew H Austern, Aart JC Bik, James C Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: a system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 135–146, 2010.

[67] Sherif Sakr, Faisal Moeen Orakzai, Ibrahim Abdelaziz, and Zuhair Khayyat. *Large-scale graph processing using Apache Giraph*. Springer, 2016.

[68] Hang Liu and H Howie Huang. {SIMD-X}: Programming and processing of graph algorithms on {GPUs}. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 411–428, 2019.

[69] Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. X-stream: Edge-centric graph processing using streaming partitions. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, pages 472–488, 2013.

[70] Joseph E Gonzalez, Reynold S Xin, Ankur Dave, Daniel Crankshaw, Michael J Franklin, and Ion Stoica. {GraphX}: Graph processing in a distributed dataflow framework. In *11th USENIX symposium on operating systems design and implementation (OSDI 14)*, pages 599–613, 2014.

[71] Miguel E Coimbra, Alexandre P Francisco, and Luís Veiga. An analysis of the graph processing landscape. *journal of Big Data*, 8(1):55, 2021.

[72] Vasiliki Kalavri, Vladimir Vlassov, and Seif Haridi. High-level programming abstractions for distributed graph processing. *IEEE Transactions on Knowledge and Data Engineering*, 30(2):305–324, 2017.

[73] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D Owens. Gunrock: A high-performance graph processing library on the gpu. In *Proceedings of the 21st ACM SIGPLAN symposium on principles and practice of parallel programming*, pages 1–12, 2016.

[74] Laxman Dhulipala, Guy Blelloch, and Julian Shun. Julienne: A framework for parallel graph algorithms using work-efficient bucketing. In *Proceedings of the 29th ACM Symposium on Parallelism in Algorithms and Architectures*, pages 293–304, 2017.

[75] Aydın Buluç and John R Gilbert. The combinatorial blas: Design, implementation, and applications. *The International Journal of High Performance Computing Applications*, 25(4):496–509, 2011.

[76] Ariful Azad, Oguz Selvitopi, Md Taufique Hussain, John R Gilbert, and Aydın Buluç. Combinatorial blas 2.0: Scaling combinatorial algorithms on distributed-memory systems. *IEEE Transactions on Parallel and Distributed Systems*, 33(4):989–1001, 2021.

[77] Tim Mattson, David Bader, Jon Berry, Aydin Buluc, Jack Dongarra, Christos Faloutsos, John Feo, John Gilbert, Joseph Gonzalez, Bruce Hendrickson, et al. Standards for graph algorithm primitives. In *2013 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–2. IEEE, 2013.

[78] Jeremy Kepner, David Bader, Aydın Buluç, John Gilbert, Timothy Mattson, and Henning Meyerhenke. Graphs, matrices, and the graphblas: Seven good reasons. *Procedia Computer Science*, 51:2453–2462, 2015.

[79] Jeremy Kepner, Peter Aaltonen, David Bader, Aydin Buluç, Franz Franchetti, John Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, et al. Mathematical foundations of the graphblas. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–9. IEEE, 2016.

[80] Narayanan Sundaram, Nadathur Satish, Md Mostofa Ali Patwary, Subramanya R Dulloor, Michael J Anderson, Satya Gautam Vadlamudi, Dipankar Das, and Pradeep Dubey. Graphmat: High performance graph analytics made productive. *Proceedings of the VLDB Endowment*, 8(11), 2015.

[81] Ariful Azad, Mohsen Mahmoudi Aznaveh, Scott Beamer, Mark Blanco, Jinhao Chen, Luke D'Alessandro, Roshan Dathathri, Tim Davis, Kevin Deweese, Jesun Firoz, et al. Evaluation of graph analytics frameworks using the gap benchmark suite. In *2020 IEEE International Symposium on Workload Characterization (IISWC)*, pages 216–227. IEEE, 2020.

[82] Nadathur Satish, Narayanan Sundaram, Md Mostofa Ali Patwary, Jiwon Seo, Jongsoo Park, M Amber Hassaan, Shubho Sengupta, Zhaoming Yin, and Pradeep Dubey. Navigating the maze of graph analytics frameworks using massive graph datasets. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*, pages 979–990, 2014.

[83] Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN symposium on Principles and practice of parallel programming*, pages 135–146, 2013.

[84] Laxman Dhulipala, Guy E Blelloch, and Julian Shun. Theoretically efficient parallel graph algorithms can be fast and scalable. *ACM Transactions on Parallel Computing (TOPC)*, 8(1):1–70, 2021.

[85] Aleksa Vukotic, Nicki Watt, Tareq Abedrabbo, Dominic Fox, and Jonas Partner. *Neo4j in action*, volume 22. Manning Shelter Island, 2015.

[86] I-Hsin Chung, Changhoan Kim, Hui-Fang Wen, and Guojing Cong. Application data prefetching on the ibm blue gene/q supercomputer. In *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, pages 1–8. IEEE, 2012.

[87] Aydin Buluç, Erika Duriakova, Armando Fox, John R Gilbert, Shoaib Kamil, Adam Lugowski, Leonid Oliker, and Samuel Williams. High-productivity and high-performance analysis of filtered semantic graphs. In *2013 IEEE 27th International Symposium on Parallel and Distributed Processing*, pages 237–248. IEEE, 2013.

[88] Abanti Basak, Shuangchen Li, Xing Hu, Sang Min Oh, Xinfeng Xie, Li Zhao, Xiaowei Jiang, and Yuan Xie. Analysis and optimization of the memory hierarchy for graph processing workloads. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 373–386. IEEE, 2019.

[89] Jóakim V Kistowski, Jeremy A Arnold, Karl Huppler, Klaus-Dieter Lange, John L Henning, and Paul Cao. How to build a benchmark. In *Proceedings of the 6th ACM/SPEC international conference on performance engineering*, pages 333–336, 2015.

[90] Jack J Dongarra, Piotr Luszczek, and Antoine Petitet. The linpack benchmark: past, present and future. *Concurrency and Computation: practice and experience*, 15(9):803–820, 2003.

[91] Holger H Hoos and Thomas Stützle. Satlib: An online resource for research on sat. *Sat*, 2000:283–292, 2000.

[92] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13*, pages 740–755. Springer, 2014.

[93] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6904–6913, 2017.

[94] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2024.

[95] Rafael H Saavedra and Alan J Smith. Analysis of benchmark characteristics and benchmark performance prediction. *ACM Transactions on Computer Systems (TOCS)*, 14(4):344–384, 1996.

[96] Aaron Stump, Geoff Sutcliffe, and Cesare Tinelli. Starexec: A cross-community infrastructure for logic solving. In *International joint conference on automated reasoning*, pages 367–373. Springer, 2014.

[97] Sewon Min, Jordan Boyd-Graber, Chris Alberti, Danqi Chen, Eunsol Choi, Michael Collins, Kelvin Guu, Hannaneh Hajishirzi, Kenton Lee, Jennimaria Palomaki, Colin Raffel, Adam Roberts, Tom Kwiatkowski, Patrick Lewis, Yuxiang Wu, Heinrich Küttler, Linqing Liu, Pasquale Minervini, Pontus Stenetorp, Sebastian Riedel, Sohee Yang, Minjoon Seo, Gautier Izacard, Fabio Petroni, Lucas Hosseini, Nicola De Cao, Edouard Grave, Ikuya Yamada, Sonse Shimaoka, Masatoshi Suzuki, Shumpei Miyawaki, Shun Sato, Ryo Takahashi, Jun Suzuki, Martin Fajcik, Martin Docekal, Karel Ondrej, Pavel Smrz, Hao Cheng, Yelong Shen, Xiaodong Liu, Pengcheng He, Weizhu Chen, Jianfeng Gao, Barlas Oguz, Xilun Chen, Vladimir Karpukhin, Stan Peshterliev, Dmytro Okhonko, Michael Schlichtkrull, Sonal Gupta, Yashar Mehdad, and Wen-tau Yih. Neurips 2020 efficientqa competition: Systems, analyses and lessons learned. In Hugo Jair Escalante and Katja Hofmann, editors, *Proceedings of the NeurIPS 2020 Competition and Demonstration Track*, volume 133 of *Proceedings of Machine Learning Research*, pages 86–111. PMLR, 06–12 Dec 2021.

[98] Richard C Murphy, Kyle B Wheeler, Brian W Barrett, and James A Ang. Introducing the graph 500. *Cray Users Group (CUG)*, 19:45–74, 2010.

[99] Julian Shun, Guy E Blelloch, Jeremy T Fineman, Phillip B Gibbons, Aapo Kyrola, Harsha Vardhan Simhadri, and Kanat Tangwongsan. Brief announcement: The problem based benchmark suite. In *Proceedings of the twenty-fourth annual ACM symposium on Parallelism in algorithms and architectures*, pages 68–70, 2012.

[100] Jure Leskovec and Rok Sosič. Snap: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology (TIST)*, 8(1):1–20, 2016.

[101] Mihai Capotă, Tim Hegeman, Alexandru Iosup, Arnau Prat-Pérez, Orri Erling, and Peter Boncz. Graphalytics: A big data benchmark for graph-processing platforms. In *Proceedings of the GRADES'15*, pages 1–6. 2015.

[102] Alexandru Iosup, Tim Hegeman, Wing Lung Ngai, Stijn Heldens, Arnau Prat-Pérez, Thomas Manhardto, Hassan Chafio, Mihai Capotă, Narayanan Sundaram, Michael Anderson, et al. Ldbc graphalytics: A benchmark for large-scale graph analysis on parallel and distributed platforms. *Proceedings of the VLDB Endowment*, 9(13):1317–1328, 2016.

[103] Scott Beamer, Krste Asanović, and David Patterson. The gap benchmark suite. *arXiv preprint arXiv:1508.03619*, 2017.

[104] Abanti Basak, Jilan Lin, Ryan Lorica, Xinfeng Xie, Zeshan Chishti, Alaa Alameldeen, and Yuan Xie. Saga-bench: Software and hardware characterization of streaming graph analytics workloads. In *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 12–23. IEEE, 2020.

[105] Laxman Dhulipala, Jessica Shi, Tom Tseng, Guy E Blelloch, and Julian Shun. The graph based benchmark suite (gbbs). In *Proceedings of the 3rd Joint International Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)*, pages 1–8, 2020.

[106] Daniel Anderson, Guy E Blelloch, Laxman Dhulipala, Magdalen Dobson, and Yihan Sun. The problem-based benchmark suite (pbbs), v2. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 445–447, 2022.

[107] Alexandru Iosup, Ahmed Musaafir, Alexandru Uta, Arnau Prat Pérez, Gábor Szárnyas, Hassan Chafi, Ilie Gabriel Tănase, Lifeng Nai, Michael Anderson, Mihai Capotă, et al. The ldbc graphalytics benchmark version 1.0.5. *arXiv preprint arXiv:2011.15028*, 2023.

[108] Ying Zhang, Pham Minh Duc, Oscar Corcho, and Jean-Paul Calbimonte. Srbench: a streaming rdf/sparql benchmark. In *The Semantic Web–ISWC 2012: 11th International Semantic Web Conference, Boston, MA, USA, November 11-15, 2012, Proceedings, Part I 11*, pages 641–657. Springer, 2012.

[109] Renzo Angles, Arnau Prat-Pérez, David Dominguez-Sal, and Josep-Lluis Larriba-Pey. Benchmarking database systems for social network applications. In *First International Workshop on Graph Data Management Experiences and Systems*, pages 1–7, 2013.

[110] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. The ldbc social network benchmark: Interactive workload. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, pages 619–630, 2015.

[111] McKnight Consulting (Sponsored by Stardog). Trillion edge knowledge graph. Technical report, Jun 2021. Accessed 04 Sep 2023, Sponsored by Stardog.

[112] Ante Javor. Memgraph vs. neo4j: A performance comparison. blog, Apr 2022. Accessed 04 Sep 23.

[113] Ante Javor. Introduction to benchgraph and its architecture. blog, Apr 2023. Accessed 04 Sep 23.

[114] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33:22118–22133, 2020.

[115] Junwei Wu, Jingwei Sun, Hao Sun, and Guangzhong Sun. Performance analysis of graph neural network frameworks. In *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 118–127. IEEE, 2021.

[116] Kezhao Huang, Jidong Zhai, Zhen Zheng, Youngmin Yi, and Xipeng Shen. Understanding and bridging the gaps in current gnn performance optimizations. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 119–132, 2021.

[117] Vijay Prakash Dwivedi, Ladislav Rampášek, Michael Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. Long range graph benchmark. *Advances in Neural Information Processing Systems*, 35:22326–22340, 2022.

[118] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. *Journal of Machine Learning Research*, 24(43):1–48, 2023.

[119] Zhiyao Zhou, Sheng Zhou, Bochao Mao, Xuanyi Zhou, Jiawei Chen, Qiaoyu Tan, Daochen Zha, Yan Feng, Chun Chen, and Can Wang. Opengsl: A comprehensive benchmark for graph structure learning. *Advances in Neural Information Processing Systems*, 36, 2024.

[120] Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, 2015.

[121] David A Bader, John Feo, John Gilbert, Jeremy Kepner, David Koester, Eugene Loh, Kamesh Madduri, Bill Mann, Theresa Meuse, and Eric Robinson. Hpc scalable graph analysis benchmark. *Citeseer. Citeseer*, 2009:1–10, 2009.

[122] Intel. A hierarchical identify verify exploit (hive) project: Programmable and integrated unified memory architecture (piuma) workload analysis document. Technical report, Intel Corporation, 2020.

[123] Eliyahu M Goldratt and Jeff Cox. *The goal: a process of ongoing improvement*. Routledge, third revised edition edition, 2004.

[124] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. A scalable processing-in-memory accelerator for parallel graph processing. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 105–117, 2015.

[125] Zhisong Fu, Michael Personick, and Bryan Thompson. Mapgraph: A high level api for fast development of high performance graph analytics on gpus. In *Proceedings of workshop on GRAph data management experiences and systems*, pages 1–6, 2014.

[126] Aydin Buluç and John R Gilbert. Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments. *SIAM Journal on Scientific Computing*, 34(4):C170–C191, 2012.

[127] Sriram Aananthakrishnan, Nesreen K Ahmed, Vincent Cave, Marcelo Cintra, Yigit Demir, Kristof Du Bois, Stijn Eyerman, Joshua B Fryman, Ivan Ganev, Wim Heirman, et al. Piuma: programmable integrated unified memory architecture. *arXiv preprint arXiv:2010.06277*, 2020.

[128] Yunming Zhang, Vladimir Kiriansky, Charith Mendis, Saman Amarasinghe, and Matei Zaharia. Making caches work for graph analytics. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 293–302. IEEE, 2017.

[129] William S Song, Vitaliy Gleyzer, Alexei Lomakin, and Jeremy Kepner. Novel graph processor architecture, prototype system, and results. In *2016 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–7. IEEE, 2016.

[130] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. Profiling a warehouse-scale computer. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 158–169, 2015.

[131] Tayo Oguntebi and Kunle Olukotun. Graphops: A dataflow library for graph analytics acceleration. In *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pages 111–117, 2016.

[132] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. Graphicionado: A high-performance and energy-efficient accelerator for graph analytics. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–13. IEEE, 2016.

[133] Anurag Mukkara, Nathan Beckmann, Maleen Abeydeera, Xiaosong Ma, and Daniel Sanchez. Exploiting locality in graph analytics through hardware-accelerated traversal scheduling. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–14. IEEE, 2018.

[134] Timothy Dysart, Peter Kogge, Martin Deneroff, Eric Bovell, Preston Briggs, Jay Brockman, Kenneth Jacobsen, Yujen Juan, Shannon Kuntz, Richard Lethin, et al. Highly scalable near memory processing with migrating threads on the emu system architecture. In *2016 6th Workshop on Irregular Applications: Architecture and Algorithms (IA3)*, pages 2–9. IEEE, 2016.

[135] Vidushi Dadu, Sihao Liu, and Tony Nowatzki. Polygraph: Exposing the value of flexibility for graph processing accelerators. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 595–608. IEEE, 2021.

[136] Muhammet Mustafa Ozdal, Serif Yesil, Taemin Kim, Andrey Ayupov, John Greth, Steven Burns, and Ozcan Ozturk. Graph analytics accelerators for cognitive systems. *IEEE Micro*, 37(1):42–51, 2017.

[137] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. Hygcn: A gcn accelerator with hybrid architecture. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 15–29. IEEE, 2020.

[138] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. *A Modern Primer on Processing in Memory*, pages 171–243. Springer Nature Singapore, Singapore, 2023.

[139] Kevin Kiningham, Philip Levis, and Christopher Ré. Grip: A graph neural network accelerator architecture. *IEEE Transactions on Computers*, 72(4):914–925, 2022.

[140] Rong Chen, Jiaxin Shi, Yanzhe Chen, Binyu Zang, Haibing Guan, and Haibo Chen. Powerlyra: Differentiated graph computation and partitioning on skewed graphs. *ACM Transactions on Parallel Computing (TOPC)*, 5(3):1–39, 2019.

[141] Luiz CR Carpinetti and Alexandre M De Melo. What to benchmark? a systematic approach and cases. *Benchmarking: An International Journal*, 9(3):244–255, 2002.

[142] Karl Huppler. The art of building a good benchmark. In *Technology conference on performance evaluation and benchmarking*, pages 18–30. Springer, 2009.

[143] Vignesh Balaji and Brandon Lucia. When is graph reordering an optimization? studying the effect of lightweight graph reordering across applications and input graphs. In *2018 IEEE International Symposium on Workload Characterization (IISWC)*, pages 203–214. IEEE, 2018.

[144] Santo Fortunato and Marc Barthelemy. Resolution limit in community detection. *Proceedings of the national academy of sciences*, 104(1):36–41, 2007.

[145] Alexander Strehl and Joydeep Ghosh. Cluster ensembles—a knowledge reuse framework for combining multiple partitions. *Journal of machine learning research*, 3(Dec):583–617, 2002.

[146] Giulio Rossetti, Luca Pappalardo, and Salvatore Rinzivillo. A novel approach to evaluate community detection algorithms on ground truth. In *Complex Networks VII: Proceedings of the 7th Workshop on Complex Networks CompleNet 2016*, pages 133–144. Springer, 2016.

[147] Giulio Rossetti. Rdyn: graph benchmark handling community dynamics. *Journal of Complex Networks*, 5(6):893–912, 2017.

[148] Giulio Rossetti and Rémy Cazabet. Community discovery in dynamic networks: a survey. *ACM computing surveys (CSUR)*, 51(2):1–37, 2018.

[149] Mark EJ Newman and Michelle Girvan. Finding and evaluating community structure in networks. *Physical review E*, 69(2):026113, 2004.

[150] Jure Leskovec, Kevin J Lang, and Michael Mahoney. Empirical comparison of algorithms for network community detection. In *Proceedings of the 19th international conference on World wide web*, pages 631–640, 2010.

[151] Vincent A Traag, Paul Van Dooren, and Yurii Nesterov. Narrow scope for resolution-limit-free community detection. *Physical Review E*, 84(1):016114, 2011.

[152] Albert-László Barabási, Réka Albert, and Hawoong Jeong. Mean-field theory for scale-free random networks. *Physica A: Statistical Mechanics and its Applications*, 272(1-2):173–187, 1999.

[153] Brian W Kernighan and Shen Lin. An efficient heuristic procedure for partitioning graphs. *The Bell system technical journal*, 49(2):291–307, 1970.

[154] Jordi Duch and Alex Arenas. Community detection in complex networks using extremal optimization. *Physical review E*, 72(2):027104, 2005.

[155] Mark EJ Newman. Modularity and community structure in networks. *Proceedings of the national academy of sciences*, 103(23):8577–8582, 2006.

[156] Bo Yang, William Cheung, and Jiming Liu. Community mining from signed social networks. *IEEE transactions on knowledge and data engineering*, 19(10):1333–1348, 2007.

[157] Jörg Reichardt and Stefan Bornholdt. Detecting fuzzy community structures in complex networks with a potts model. *Physical review letters*, 93(21):218701, 2004.

[158] Andrew Ng, Michael Jordan, and Yair Weiss. On spectral clustering: Analysis and an algorithm. *Advances in neural information processing systems*, 14, 2001.

[159] Scott White and Padhraic Smyth. A spectral clustering approach to finding communities in graphs. In *Proceedings of the 2005 SIAM international conference on data mining*, pages 274–285. SIAM, 2005.

[160] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. Finding community structure in very large networks. *Physical review E*, 70(6):066111, 2004.

[161] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment*, 2008(10):P10008, 2008.

[162] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9(1):5233, 2019.

[163] Liang Yang, Xiaochun Cao, Dongxiao He, Chuan Wang, Xiao Wang, and Weixiong Zhang. Modularity based community detection with deep learning. In *IJCAI*, volume 16, pages 2252–2258, 2016.

[164] Sandro Cavallari, Vincent W Zheng, Hongyun Cai, Kevin Chen-Chuan Chang, and Erik Cambria. Learning community embedding with community detection and node embedding on graphs. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pages 377–386, 2017.

[165] Xiao Wang, Peng Cui, Jing Wang, Jian Pei, Wenwu Zhu, and Shiqiang Yang. Community preserving network embedding. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.

[166] Nagehan İlhan and Şule Gündüz Öğüdücü. Predicting community evolution based on time series modeling. In *Proceedings of the 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining 2015*, pages 1509–1516, 2015.

[167] Shweta Bansal, Sanjukta Bhowmick, and Prashant Paymal. Fast community detection for dynamic complex networks. In *Complex Networks: Second International Workshop, CompleNet 2010, Rio de Janeiro, Brazil, October 13-15, 2010, Revised Selected Papers*, pages 196–207. Springer, 2011.

[168] Anne-Sophie Himmel, Hendrik Molter, Rolf Niedermeier, and Manuel Sorge. Enumerating maximal cliques in temporal graphs. In *2016 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pages 337–344. IEEE, 2016.

[169] Jörg Reichardt and Stefan Bornholdt. Statistical mechanics of community detection. *Physical review E*, 74(1):016110, 2006.

[170] Martin Rosvall and Carl T Bergstrom. Maps of random walks on complex networks reveal community structure. *Proceedings of the national academy of sciences*, 105(4):1118–1123, 2008.

[171] Zhong-Yuan Zhang. Community structure detection in complex networks with partial background information. *Europhysics Letters*, 101(4):48005, 2013.

[172] Yuduo Wu, Yangzihao Wang, Yuechao Pan, Carl Yang, and John D Owens. Performance characterization of high-level programming models for gpu graph analytics. In *2015 IEEE International Symposium on Workload Characterization*, pages 66–75. IEEE, 2015.

[173] Shichang Zhang, Atefeh Sohrabizadeh, Cheng Wan, Zijie Huang, Ziniu Hu, Yewen Wang, Jason Cong, Yizhou Sun, et al. A survey on graph neural network acceleration: Algorithms, systems, and customized hardware. *arXiv preprint arXiv:2306.14052*, 2023.

[174] Yi-Chien Lin and Viktor Prasanna. Hyscale-gnn: A scalable hybrid gnn training system on single-node heterogeneous architecture. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 557–567. IEEE, 2023.

[175] Bingyi Zhang and Viktor Prasanna. Dynasparse: Accelerating gnn inference through dynamic sparsity exploitation. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 233–244. IEEE, 2023.

[176] Yi-Chien Lin, Bingyi Zhang, and Viktor Prasanna. Hitgnn: High-throughput gnn training framework on cpu+ multi-fpga heterogeneous platform. *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[177] Zeyu Zhu, Fanrong Li, Gang Li, Zejian Liu, Zitao Mo, Qinghao Hu, Xiaoyao Liang, and Jian Cheng. Mega: A memory-efficient gnn accelerator exploiting degree-aware mixed-precision quantization, 2023.

[178] Kai Zhong, Shulin Zeng, Wentao Hou, Guohao Dai, Zhenhua Zhu, Xuecang Zhang, Shihai Xiao, Huazhong Yang, and Yu Wang. Cognn: An algorithm-hardware co-design approach to accelerate gnn inference with mini-batch sampling. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.

[179] Lizhi Zhang, Kai Lu, Zhiquan Lai, Yongquan Fu, Yu Tang, and Dongsheng Li. Accelerating gnn training by adapting large graphs to distributed heterogeneous architectures. *IEEE Transactions on Computers*, 2023.

[180] Jiaxian Chen, Zhaoyu Zhong, Kaoyi Sun, Chenlin Ma, Rui Mao, and Yi Wang. Lift: Exploiting hybrid stacked memory for energy-efficient processing of graph convolutional networks. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2023.

[181] Stefan Abi-Karam and Cong Hao. Gnnbuilder: An automated framework for generic graph neural network accelerator generation, simulation, and optimization. In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*, pages 212–218. IEEE, 2023.

[182] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, Hao Lu, Daniel Chavarrià-Miranda, Arif Khan, and Assefaw Gebremedhin. Distributed louvain algorithm for graph community detection. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 885–895, 2018.

[183] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, and Ananth Kalyanarainan. Scaling and quality of modularity optimization methods for graph clustering. In *2019 IEEE High Performance Extreme Computing Conference (HPEC)*, pages 1–6. IEEE, 2019.

[184] Tze Meng Low, Daniele G Spampinato, Scott McMillan, and Michel Pelletier. Linear algebraic louvain method in python. In *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 223–226. IEEE, 2020.

[185] Tim Mattson, Timothy A Davis, Manoj Kumar, Aydin Buluc, Scott McMillan, José Moreira, and Carl Yang. Lagraph: A community effort to collect graph algorithms built on top of the graphblas. In *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 276–284. IEEE, 2019.

[186] Aric Hagberg, Pieter Swart, and Daniel S Chult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.

[187] Andrea Lancichinetti, Santo Fortunato, and Filippo Radicchi. Benchmark graphs for testing community detection algorithms. *Physical review E*, 78(4):046110, 2008.

[188] Seiji Maekawa, Jianpeng Zhang, George Fletcher, and Makoto Onizuka. General generator for attributed graphs with community structure. In *Proceeding of the ECML/PKDD graph embedding and mining workshop*, pages 1–5, 2019.

[189] Seiji Maekawa, Yuya Sasaki, George Fletcher, and Makoto Onizuka. Gencat: Generating attributed graphs with controlled relationships between classes, attributes, and topology. *Information Systems*, 115:102195, 2023.

[190] Omid Madani, Sai Ankith Averineni, and Shashidhar Gandham. A dataset of networks of computing hosts. In *Proceedings of the 2022 ACM on International Workshop on Security and Privacy Analytics*, pages 100–104, 2022.

[191] Hao Yin, Austin R Benson, Jure Leskovec, and David F Gleich. Local higher-order graph clustering. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 555–564, 2017.

[192] Jaewon Yang and Jure Leskovec. Defining and evaluating network communities based on ground-truth. In *Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics*, pages 1–8, 2012.

[193] Gerrit Lahrmann, Frederik Marx, Tobias Mettler, Robert Winter, and Felix Wortmann. Inductive design of maturity models: applying the rasch algorithm for design science research. In *Service-Oriented Perspectives in Design Science Research: 6th International Conference, DESRIST 2011, Milwaukee, WI, USA, May 5-6, 2011. Proceedings 6*, pages 176–191. Springer, 2011.

[194] Raghad Baker Sadiq, Nurhizam Safie, Abdul Hadi Abd Rahman, and Shidrokh Goudarzi. Artificial intelligence maturity model: a systematic literature review. *PeerJ Computer Science*, 7:e661, 2021.